Wanessa Koserski dos Santos

Análise de fatores de otimização em transpilação quântica utilizando aprendizado supervisionado

São Paulo - Brasil

2025

Wanessa Koserski dos Santos

Análise de fatores de otimização em transpilação quântica utilizando aprendizado supervisionado

Monografia apresentada na disciplina Trabalho de Conclusão de Curso, como parte dos requisitos para obtenção do título de Bacharel em Ciência da Computação.

Centro Universitário Senac - Santo Amaro
Bacharelado em Ciência da Computação

Orientador: Jorge de Oliveira Echeimberg

São Paulo - Brasil
2025

"Deus às vezes joga dados,
onde ninguém pode ver."
(HAWKING, Stephen)

Resumo

Este trabalho investiga a caracterização de fatores que influenciam a otimização de circuitos quânticos durante o processo de transpilação com modelos de aprendizado supervisionado baseados na arquitetura Transformer. Para isso, foram utilizados circuitos do MQT Bench como base de treinamento, permitindo que o modelo aprendesse a mapear versões não otimizadas para versões otimizadas geradas pelo qiskit. Ao longo do desenvolvimento, diferentes configurações de função de perda foram avaliadas para identificar como o modelo prioriza métricas como profundidade, contagem de portas, fidelidade e portas de dois *qubits*. Os resultados indicam que o Transformer é capaz de capturar padrões relevantes de otimização, apresentando desempenho competitivo com o qiskit em diversos cenários, especialmente na redução de portas *CNOT* e portas *multiqubit*. Contudo, o modelo apresentou alta variância e não garante equivalência lógica explícita, o que limita seu uso em produção. Os experimentos em *hardware* real reforçam que, apesar dessas limitações, a maioria das otimizações preserva comportamentos próximos aos obtidos pelo qiskit.

Palavras-chaves: computação quântica, transpilação, otimização, aprendizado supervisionado, Transformer.

Abstract

This work investigates the characterization of factors influencing the optimization of quantum circuits during the transpilation process using supervised learning models based on the Transformer architecture. For this purpose, circuits from the MQT Bench were used as a training base, allowing the model to learn to map unoptimized versions to optimized versions generated by qiskit. Throughout the development, different loss function configurations were evaluated to identify how the model prioritizes metrics such as depth, gate count, fidelity, and two-qubit gates. The results indicate that Transformer is capable of capturing relevant optimization patterns, showing competitive performance with qiskit in several scenarios, especially in the reduction of CNOT gates and multiqubit gates. However, the model presented high variance and does not guarantee explicit logical equivalence, which limits its use in production. Experiments on real hardware reinforce that, despite these limitations, most optimizations preserve behaviors close to those obtained by qiskit.

Key-words: quantum computing, transpilation, optimization, supervised learning, Transformer.

Lista de ilustrações

Figura 1 – Experimento da fenda dupla de Thomson Young 14
Figura 2 – Exemplo de circuito quântico com portas básicas aplicadas a dois qubits 18
Figura 3 – Exemplo de porta *CNOT* 18
Figura 4 – Exemplo de circuito Bell 19
Figura 5 – Exemplo de porta Toffoli 19
Figura 6 – Resumo do fluxo da transpilação quântica 23

Lista de tabelas

Tabela 1 – Portas quânticas comuns de referência 17
Tabela 2 – Características de três classes de circuitos quânticos amplamente utilizados 20
Tabela 3 – Evolução dos pesos da função de perda entre versões 38
Tabela 4 – Transformer vs Qiskit (Version 1.0) 40
Tabela 5 – Transformer vs Qiskit (Version 2.0) 40
Tabela 6 – Transformer vs Qiskit (Version 3.0) 40
Tabela 7 – Transformer vs Qiskit (Version 4.0) 41
Tabela 8 – Transformer vs Qiskit (Version 5.0) 41
Tabela 9 – Comparação de desempenho entre versões 41
Tabela 10 – Estratégia de treinamento e seu impacto 42

Lista de abreviaturas e siglas

bit	Binary digit
CCNOT	*Controlled Controlled NOT* (controlador do *NOT* controlado)
CNOT	*Controlled NOT* (*NOT* controlado)
CLI	*Command-Line Interface* (Interface de Linha de Comando)
CX	*Controlled X*
GHZ	Greenberger-Horne-Zeilinger
IA	Inteligência Artificial
IBM	*International Business Machines* (Máquinas empresariais internacionais)
laser	*Light Amplification by Stimulated Emission of Radiation* (Amplificação da luz por emissão estimulada de radiação)
ML	*Machine Learning* (Aprendizado de Máquina)
	MQT *Munich Quantum Toolkit*
NISQ	*Noisy Intermediate Scale Quantum* (Quântica ruidosa de escala intermediária)
	QASM *Open Quantum Assembly Language*
QAOA	*Quantum Approximate Optimization Algorithm* (Algoritmo de otimização aproximada quântica)
QFT	*Quantum Fourier Transform* (Transforamada de Fourir quântica)
Qiskit	*Quantum Information Science Kit* (Kit de ciência de informação quântica)
	QPY *Qiskit Python Serialization* (Serealização Qiskit Python)
qubit	*Quantum Bit*
RL	*Reinforcement Learning* (Aprendizagem por reforço)
SDK	*Software Development Kit* (Kit de desenvolvimento de software)

Sumário

1 INTRODUÇÃO . . . 10
1.1 Contexto . . . 10
1.2 Objetivos . . . 11
1.2.1 Objetivo Geral . . . 11
1.2.2 Objetivos Específicos . . . 11

2 REVISÃO BIBLIOGRÁFICA . . . 12
2.1 Conceitos básicos de computação quântica . . . 12
2.1.1 Conceitos matemáticos utilizados . . . 12
2.1.1.1 Notação de Dirac . . . 12
2.1.1.2 Produto tensorial . . . 12
2.1.2 Conceitos da física quântica utilizados . . . 13
2.1.2.1 Superposição . . . 13
2.1.2.2 Emaranhamento ou entrelaçamento . . . 14
2.1.3 *Quantum bit* ou *qubit* . . . 14
2.1.4 Portas quânticas . . . 15
2.1.5 Circuitos quânticos . . . 17
2.1.6 Portas e circuitos nativos do *hardware* quântico . . . 20
2.1.6.1 Ruído . . . 21
2.2 Transpilação . . . 22
2.2.1 Passes e fluxo da transpilação . . . 22
2.2.1.1 Roteamento dos *qubits* . . . 23
2.2.1.2 Decomposição de portas . . . 24
2.2.1.3 Otimização e suas métricas . . . 24
2.3 Inteligência artificial . . . 25
2.3.1 Aprendizado de máquina . . . 25
2.3.1.1 Aprendizagem supervisionada . . . 26
2.3.1.2 Transformer . . . 26
2.4 Ferramentas . . . 27
2.4.1 Pytorch . . . 27
2.4.2 Qiskit . . . 29
2.4.3 IBM Quantum . . . 30
2.4.4 MQT Bench . . . 30
2.5 Trabalhos relacionados . . . 31

3 DESENVOLVIMENTO . . . 33
3.1 Metodologia . . . 33
3.2 Dados Utilizados . . . 33
3.3 Processo de Treinamento . . . 35
3.4 Processo de Testes . . . 39
3.5 Resultados . . . 41

4 CONCLUSÃO . . . 44
4.1 Trabalhos Futuros . . . 44

REFERÊNCIAS 45

APÊNDICES 48

APÊNDICE A – MODEL.PY 49

APÊNDICE B – TOKENIZER.PY 53

APÊNDICE C – TRAINER.PY - V5.0 57

APÊNDICE D – METRICS.PY 65

APÊNDICE E – TRAIN_MODEL.PY 69

APÊNDICE F – TEST_REAL_HARDWARE_IBM.PY 77

1 Introdução

Este trabalho aborda métodos de transpilação em hardware quântico, com foco na otimização de circuitos e no uso de modelos de inteligência artificial para aprender padrões utilizados por transpiladores. A proposta consiste em analisar como características estruturais de circuitos quânticos influenciam o processo de otimização, especialmente em dispositivos NISQ, nos quais o ruído, a conectividade limitada e as taxas de erro de portas afetam diretamente a fidelidade dos resultados.

A estrutura deste trabalho está divida em: introdução, na qual o tópico é contextualizado e os objetivos delineados; uma revisão bibliográfica, compreendendo a pesquisa realizada para fundamentar todas as afirmações; e uma seção de desenvolvimento, como os componentes subsequentes deste estudo serão organizados.

1.1 Contexto

A computação quântica tem o potencial de aumentar a capacidade computacional e transformar como é abordado a solução de problemas. (RAKYTA; ZIMBORáS, 2021) Isso se deve ao fato da utilização de princípios da mecânica quântica, como superposição, interferência e emaranhamento, para alcançar um nível de paralelismo e poder computacional que supera as capacidades da computação clássica (HIDARY, 2019).

A otimização é frequentemente mencionada como uma área que tem um ganho significativo, possuindo a capacidade de melhorar a qualidade das soluções, aumentar a diversidade, diminuir o tempo necessário para soluções e reduzir as despesas relacionadas (ABBAS et al., 2023).

As aplicações em potencial são extensas e estão em expansão, incluindo campos como simulações moleculares, cibersegurança, finanças e logística. Além disso, a computação quântica não é utilizada apenas para aprimorar os algoritmos clássicos de aprendizado de máquina, mas também para construir novos algoritmos quânticos de aprendizado de máquina (RAMALHO; SOUZA; CHAIM, 2025).

No entanto, os processadores quânticos atualmente disponíveis, frequentemente referidos como dispositivos de escala intermediária ruidosa (NISQ - *Noisy Intermediate Scale Quantum*), operam com *qubits* ruidosos e conectividade limitada, impondo restrições significativas à fidelidade dos programas executados. A execução bem-sucedida de um algoritmo quântico requer que o circuito lógico correspondente seja adaptado às características físicas da arquitetura do *hardware* quântico alvo, este processo de adaptação é conhecido como transpilação.

A síntese otimizada complementa a transpilação, buscando reduzir o número de portas e a profundidade do circuito, melhorando assim sua robustez em ambientes ruidosos (GE et al., 2024b). Técnicas avançadas, como mapeamento adaptativo ao ruído e seleção dinâmica de passes de otimização, têm demonstrado potencial para aumentar a fidelidade de execução. Entretanto, a falta de uma arquitetura quântica padronizada e a natureza específica de cada aplicação dificultam o desenvolvimento de soluções universais, exigindo métodos flexíveis e adaptativos (DANGWAL et al., 2023).

Além da análise estática de passes de transpilação, abordagens baseadas em aprendizado supervisionado surgem como solução promissora, aprendendo a mapear características do circuito para ações de otimização ideais a partir de dados de treinamento. Isso permite superar as limitações de métodos exaustivos e heurísticas manuais, alcançando melhores resultados de forma mais eficiente (DANGWAL et al., 2023).

1.2 Objetivos

Os objetivos deste trabalho são organizados em um objetivo geral, responsável por orientar a investigação central, e objetivos específicos, que detalham as etapas necessárias para compreender como diferentes fatores influenciam o processo de otimização na transpilação quântica.

1.2.1 Objetivo Geral

Investigar de que forma modelos baseados na arquitetura *Transformer* são capazes de aprender padrões de otimização aplicados na transpilação quântica e analisar como características estruturais dos circuitos influenciam esse processo em dispositivos NISQ.

1.2.2 Objetivos Específicos

- Implementar, treinar e avaliar modelos supervisionados capazes de mapear circuitos originais para versões otimizadas segundo padrões produzidos pelo qiskit.
- Avaliar o desempenho do modelo em *hardware* real, comparando suas otimizações entre o qiskit e entre os modelo treinados.
- Analisar através dos testes as métricas que afetam a transpilação.

2 Revisão Bibliográfica

2.1 Conceitos básicos de computação quântica

2.1.1 Conceitos matemáticos utilizados

2.1.1.1 Notação de Dirac

De acordo com o conceito apresentado por Schneider, Burgholzer e Wille (2022), a Notação de Dirac, também chamada de notação *bra-ket*, foi criada pelo físico Paul Dirac e adotada pela física quântica devido ao fornecimento de uma estrutura matemática padronizada para simbolizar os conceitos abstratos da mecânica quântica. Particularmente, representar os estados quânticos representados como vetores em espaços complexos, as operações realizadas nesses estados e a conexão entre amplitudes e probabilidades.

O símbolo fundamental para descrever um estado quântico como um vetor nessa notação é o "ket", escrito como $|\ \rangle$, que é essencialmente um vetor de coluna, contendo rótulos que identificam um determinado estado, que podem ser símbolos, letras, números e até palavras. Por exemplo, para um qubit (equivalante ao *bit* clássico na computação quântica), os dois estados básicos que correspondem aos estados clássicos 0 e 1 são representados pelos kets $|0\rangle$ e $|1\rangle$. (SCHNEIDER; BURGHOLZER; WILLE, 2022)

Na representação vetorial, esses kets correspondem aos vetores coluna:

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \tag{2.1}$$

$$|1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \tag{2.2}$$

Por outro lado o "bra", escrito como $\langle\ |$, é a transposta conjugada do ket correspondente.

$$\langle 0| = \begin{bmatrix} 1 & 0 \end{bmatrix} \tag{2.3}$$

$$\langle 1| = \begin{bmatrix} 0 & 1 \end{bmatrix} \tag{2.4}$$

2.1.1.2 Produto tensorial

Sendo o produto tensorial definido por Silva (2024) como uma operação algébrica que combina vetores ou matrizes para formar novos objetos matemáticos em um espaço de dimensão superior. O símbolo utilizado para representar essa operação é o operador $\otimes$ e o produto tensorial entre dois elementos A e B representado como $A \otimes B$.

$$A \otimes B = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \otimes \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} a \begin{vmatrix} e & f \\ g & h \end{vmatrix} & b \begin{vmatrix} e & f \\ g & h \end{vmatrix} \\ c \begin{vmatrix} e & f \\ g & h \end{vmatrix} & d \begin{vmatrix} e & f \\ g & h \end{vmatrix} \end{bmatrix} = \begin{bmatrix} a \cdot e & a \cdot f & b \cdot e & b \cdot f \\ a \cdot g & a \cdot h & b \cdot g & b \cdot h \\ c \cdot e & c \cdot f & d \cdot e & d \cdot f \\ c \cdot g & c \cdot h & d \cdot g & d \cdot h \end{bmatrix} \tag{2.5}$$

Na computação quântica, o produto tensorial é essencial porque ele permite unir dois sistemas em um só. Quando temos dois estados quânticos separados, podemos combiná-los por meio do produto tensorial de seus vetores, criando um estado único que representa os dois ao mesmo tempo (YANOFSKY; MANNUCCI, 2008).

O autor Silva (2024) traz um exemplo de uma porta quântica *Control-X Gate* (*CX* ou *CNOT*), que tem como função quando o estado é 0 não deve fazer nada e quando o estado é 1 deve aplicar a porta X.

- Projetor de controle (filtro que só deixa passar um estado): $|0\rangle\langle 0|$ e $|1\rangle\langle 1|$
- Matriz identidade (mantem o estado): $I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$
- Porta *NOT* (inverte o estado): $X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$

$$CX = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes X = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \otimes \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \otimes \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \tag{2.6}$$

$$CX = \begin{bmatrix} 1 \begin{vmatrix} 1 & 0 \\ 0 & 1 \end{vmatrix} & 0 \begin{vmatrix} 1 & 0 \\ 0 & 1 \end{vmatrix} \\ 0 \begin{vmatrix} 1 & 0 \\ 0 & 1 \end{vmatrix} & 0 \begin{vmatrix} 1 & 0 \\ 0 & 1 \end{vmatrix} \end{bmatrix} + \begin{bmatrix} 0 \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} & 0 \begin{vmatrix} 0 & 1 \\ 1 & 0 \end{vmatrix} \\ 0 \begin{vmatrix} 0 & 1 \\ 1 & 0 \end{vmatrix} & 1 \begin{vmatrix} 0 & 1 \\ 1 & 0 \end{vmatrix} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \tag{2.7}$$

2.1.2 Conceitos da física quântica utilizados

2.1.2.1 Superposição

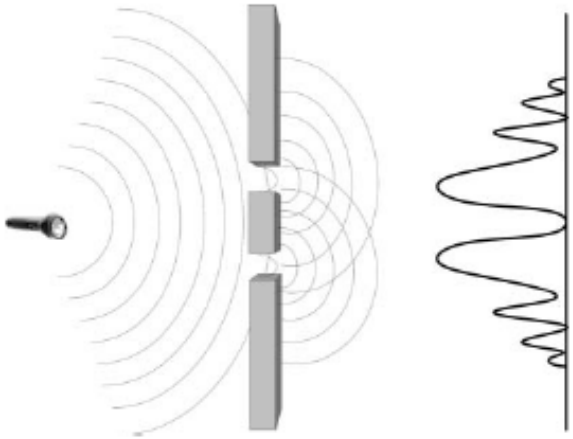
A superposição quântica é um dos princípios da mecânica quântica, descrevendo a capacidade de um sistema quântico de existir em múltiplos estados simultaneamente até que uma medição seja realizada. Esse fenômeno apresenta uma grande divergência da física clássica, em que um objeto possui atributos definidos com precisão em qualquer instante específico (BAAQUIE; KWEK, 2023).

Uma das ilustrações esclarecedoras da superposição citadas por Baaquie e Kwek (2023) é o experimento de dupla fenda, que foi conduzido primeiramente por Thomas Young por volta de 1800, que exemplifica o conceito de dualidade onda-partícula.

Quando partículas quânticas, como elétrons, são projetadas em direção a uma barreira com duas aberturas, um padrão de interferência surge na tela de detecção, que é

característico do comportamento ondulatório, como na Figura 1. Esse fenômeno indica que cada partícula quântica não atravessa apenas uma abertura, em vez disso, ela se propaga por ambas simultaneamente, existindo em uma condição de superposição.

Figura 1 – Experimento da fenda dupla de Thomson Young



Fonte: Yanofsky e Mannucci (2008)

No entanto, quando um detector é utilizado para determinar por qual fenda a partícula passa, o padrão de onda desaparece, e se comportam como um material granular. Assim, evidenciando o colapso da função de onda, no qual a medição destrói a superposição, forçando o sistema a assumir um estado definido.

2.1.2.2 Emaranhamento ou entrelaçamento

O emaranhamento ou entrelaçamento quântico é o conceito de um fenômeno em que duas ou mais partículas tornam-se fortemente correlacionadas, de modo que o estado de uma não pode ser descrito independentemente das outras, mesmo quando separadas por grandes distâncias. Isso implica que a medição do estado de uma determina imediatamente o estado da outra independentemente da distância (BAAQUIE; KWEK, 2023).

Entretanto, é importante destacar que o emaranhamento não é um envio de sinais, não transmite informação e não permite comunicação instantânea. As correlações observadas são compatíveis com a relatividade, uma vez que não podem ser utilizadas para transportar informação mais rapidamente do que a luz. Assim, embora pareçam instantâneas, tais correlações não violam o limite relativístico de velocidade.

2.1.3 *Quantum bit* ou *qubit*

De acordo com a definição dada por Yanofsky e Mannucci (2008), um *bit* é uma unidade de informação que caracteriza um sistema clássico bidimensional, cujo o conjunto de estados é composto por dois elementos. Normalmente, denotamos esses dois estados potenciais como 0 e 1 ou verdadeiro e falso.

Notação de Dirac que ilustra a forma de um *bit* como exemplo:

$$|0\rangle = \begin{matrix} \mathbf{0} \\ \mathbf{1} \end{matrix} \quad \begin{bmatrix} 1 \\ 0 \end{bmatrix} \tag{2.8}$$

É representado acima o 0, ou seja, o estado $|0\rangle$, com uma matriz 2 por 1 que contém um 1 na linha correspondente ao valor 0 e um 0 na linha correspondente ao valor 1.

Enquanto na computação clássica os estados 0 e 1 representam a ausência ou presença de fluxo de energia (como em um circuito aberto ou fechado), na computação quântica, um *qubit* indica a probabilidade da rotação (*spin*) de um elétron, quando medido, estar orientado para cima (*Spin Up* $|\uparrow\rangle$) ou para baixo (*Spin Down* $|\downarrow\rangle$) (YANOFSKY; MANNUCCI, 2008).

$$|0\rangle = |\uparrow\rangle \, e \, |1\rangle = |\downarrow\rangle \tag{2.9}$$

Além disso, no mundo clássico, um sistema só pode ocupar um estado bem definido por vez — como um interruptor que está ou ligado (1) ou desligado (0). Já na mecânica quântica, um sistema pode existir em uma sobreposição de estados, ocupando múltiplas condições simultaneamente. Por exemplo, um "interruptor quântico"poderia estar parcialmente ligado e parcialmente desligado ao mesmo tempo, até que uma medição seja realizada, colapsando-o em um dos estados clássicos (0 ou 1) (YANOFSKY; MANNUCCI, 2008).

Em termos matemáticos, essa característica é expressa por meio da superposição quântica: um *qubit* pode ser descrito como uma combinação linear dos estados básicos $|0\rangle$ e $|1\rangle$ (utilizando a notação de Dirac):

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle \tag{2.10}$$

Onde α e β são coeficientes associados as amplitudes de probabilidade complexas, com $|\alpha|^2$ e $|\beta|^2$ correspondendo às probabilidades de medir o *qubit* no estado $|0\rangle$ ou $|1\rangle$, respectivamente. Sendo necessária respeitar a propriedade matemática de: $|\alpha|^2 + |\beta|^2 = 1$ (soma de todas as probabilidades precisa resultar em 100%, ou seja, 1).

$$|\psi\rangle = \alpha \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \beta \begin{bmatrix} 0 \\ 1 \end{bmatrix} \tag{2.11}$$

$$|\psi\rangle = \begin{bmatrix} \alpha \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ \beta \end{bmatrix} \tag{2.12}$$

$$|\psi\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix} \tag{2.13}$$

2.1.4 Portas quânticas

Portas quânticas são os componentes fundamentais utilizados para construir circuitos quânticos. Elas representam as operações que atuam nos *qubits*, que são as unidades de informação quântica (YANOFSKY; MANNUCCI, 2008).

Na computação quântica, as portas quânticas são expressas matematicamente como matrizes, uma vez que os estados quânticos (*qubits*) são representados como vetores e as operações (portas) estão associadas a transformações lineares que operam nesses vetores (YANOFSKY; MANNUCCI, 2008). Os valores nas matrizes são ditados pela operação ou

transformação específica que cada porta deve executar em estados quânticos, garantindo consistentemente características unitárias para manter as probabilidades (SILVA, 2024).

Consideramos, por exemplo, uma operação que faz a transição do estado $|0\rangle$ para o estado $|1\rangle$ e vice-versa (NIELSEN; CHUANG, 2011). Essa operação está presente tanto na computação quântica quanto na clássica e é conhecida como *NOT*.

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle \tag{2.14}$$

$$|\psi\rangle = \alpha\,|1\rangle + \beta\,|0\rangle \tag{2.15}$$

Utilizando conveniente a representação da porta quântica em forma matricial, que decorre diretamente da linearidade das portas quânticas, temos assim a porta X (NIELSEN; CHUANG, 2011).

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \tag{2.16}$$

Dessa forma, a relação entre o *qubit* e a porta quântica é estabelecida por meio de cálculos matriciais. Para reformular, a matriz que representa as probabilidades dos estados que o *qubit* pode assumir é alterada pela porta quântica, resultando no surgimento de novas probabilidades sobre esse *qubit*.

$$X \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \beta \\ \alpha \end{bmatrix} \tag{2.17}$$

A condição para que uma matriz represente uma porta quântica é a unitariedade. Para uma matriz U descrever corretamente a operação de um único qubit, ela deve satisfazer: $U^{\dagger}U = I$, onde $U^{\dagger}$ é a adjunta de U (obtida pela transposição seguida da conjugação complexa) e I é a matriz identidade (NIELSEN; CHUANG, 2011).

Como demonstrado por Nielsen e Chuang (2011), uma porta unitária de um único qubit arbitrário pode ser decomposta como um produto de rotações e uma porta conhecida como rotação em torno do eixo z junto com um multiplicador constante da forma $e^{i\alpha}$. Ou seja, qualquer porta quântica de um único qubit pode ser decomposta em: $U = e^{i\alpha}R_z(\beta)R_y(\gamma)R_z(\delta)$. Sendo:

$$R_z(\beta) = \begin{bmatrix} e^{-i\frac{\beta}{2}} & 0 \\ 0 & e^{i\frac{\beta}{2}} \end{bmatrix} \tag{2.18}$$

$$R_y(\gamma) = \begin{bmatrix} \cos\frac{\gamma}{2} & -\sin\frac{\gamma}{2} \\ \sin\frac{\gamma}{2} & \cos\frac{\gamma}{2} \end{bmatrix} \tag{2.19}$$

$$R_z(\beta) = \begin{bmatrix} e^{-i\frac{\beta}{2}} & 0 \\ 0 & e^{i\frac{\beta}{2}} \end{bmatrix} \tag{2.20}$$

A seguir, é apresentada uma tabela contendo algumas das portas quânticas mais comuns, usando como base as portas citadas por Silva (2024) e Khan et al. (2024).

Abaixo está uma tabela que contém algumas portas comuns com informações trazidas pelo Silva (2024) e Khan et al. (2024).

Tabela 1 – Portas quânticas comuns de referência

Porta	**Número de Qubits**	**Efeito**	**Matriz**
Pauli-X (X)	1	Inverte o estado lógico do *qubit*.	$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$
Pauli-Z (Z)	1	Introduz uma fase -1 no estado $\|1\rangle$.	$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$
Pauli-Y (Y)	1	Combina inversão com fase imaginária.	$\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$
Hadamard (H)	1	Cria superposição e transforma entre as bases computacional e de Hadamard.	$\frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$
Fase (S)	1	Aplica fase de $\pi/2$ ao estado $\|1\rangle$.	$\begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$
T (raiz da Fase)	1	Introduz fase $\pi/4$ (importante para universalidade).	$\begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$
Rotação $R_z(\theta)$	1	Rotação em torno do eixo Z do espaço de Bloch.	$\begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}$
Rotação $R_y(\theta)$	1	Rotação em torno do eixo Y.	$\begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix}$
CNOT (CX)	2	Aplica X no alvo quando o controle é $\|1\rangle$ (gera emaranhamento).	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$
SWAP	2	Troca os estados de dois qubits.	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$
Toffoli (CCNOT)	3	Aplica X no alvo quando ambos controles estão em $\|1\rangle$.	$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$

2.1.5 Circuitos quânticos

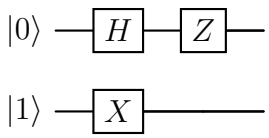
Um circuito quântico é uma representação fundamental de algoritmos em computação quântica, composto por uma sequência ordenada de operações básicas conhecidas

como portas quânticas, as quais são executadas diretamente no *hardware* quântico. Assim como ocorre na computação clássica, onde algoritmos de alto nível precisam ser traduzidos para linguagem de máquina, na computação quântica os algoritmos abstratos devem ser convertidos em operações específicas compatíveis com o dispositivo físico utilizado (SCHNEIDER; BURGHOLZER; WILLE, 2022).

A estrutura básica de um circuito quântico é composta por dois elementos essenciais: os fios quânticos, que representam os *qubits* (as unidades fundamentais de informação quântica), e as portas quânticas elementares, que operam sobre esses *qubits*, permitindo a manipulação ou transmissão de informação quântica ao longo do sistema (KARUPPASAMY et al., 2025). Em diagramas convencionais, esses componentes são representados por linhas horizontais indicando os *qubits* e por caixas rotuladas que indicam as portas aplicadas, obedecendo a uma ordem temporal que evolui da esquerda para a direita (YANOFSKY; MANNUCCI, 2008).

A Figura 2 apresenta um exemplo simples dessa representação visual, ilustrando portas quânticas elementares aplicadas a dois *qubits* ao longo do tempo.

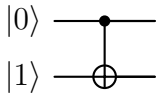
Figura 2 – Exemplo de circuito quântico com portas básicas aplicadas a dois qubits



Fonte: Criado pelo autor

Além de portas individuais, circuitos quânticos também podem incluir **portas controladoras**, que operam sobre um *qubit* somente quando outro *qubit* atua como controle. A Figura 3 exemplifica esse tipo de operação utilizando uma porta *CNOT*, um dos blocos fundamentais para a construção de algoritmos quânticos mais complexos (SCHNEIDER; BURGHOLZER; WILLE, 2022).

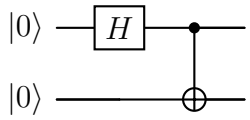
Figura 3 – Exemplo de porta *CNOT*



Fonte: Criado pelo autor

Como exemplo de circuito conhecido e amplamente utilizado é o circuito responsável pela geração de um estado de Bell, que demonstra como portas elementares podem criar emaranhamento entre dois *qubits*. O circuito correspondente é mostrado a seguir:

Figura 4 – Exemplo de circuito Bell



Fonte: Criado pelo autor

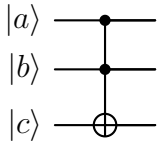
O circuito apresentado na Figura 4 aplica inicialmente a porta de Hadamard ao primeiro *qubit* e, em seguida, utiliza uma porta *CNOT* para estabelecer a correlação quântica entre ambos, resultando no estado emaranhado $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ (SCHNEIDER; BURGHOLZER; WILLE, 2022).

Do ponto de vista matemático, qualquer circuito quântico completo pode ser representado por uma única matriz unitária. Isso ocorre porque cada porta quântica individual corresponde a uma matriz unitária, e a aplicação sequencial de portas em um circuito equivale à multiplicação sucessiva dessas matrizes. Além disso, quando portas atuam sobre *qubits* diferentes no mesmo instante do circuito, a operação combinada é descrita pelo produto tensorial das matrizes correspondentes (YOUNIS; IANCU, 2022).

Assim, a matriz final associada ao circuito é construída pela composição entre produtos matriciais (para operações sequenciais) e produtos tensoriais (para operações paralelas e extensões de operadores para múltiplos qubits). Essa estrutura garante que o circuito como um todo seja sempre unitário, preservando normas e probabilidades.

A unitariedade do circuito está diretamente ligada à sua reversibilidade: como matrizes unitárias possuem inversas unitárias, toda operação quântica pode ser "desfeita" por meio da sua matriz adjunta. Diferente de como é encontrado na computação clássica, onde operações como *AND*, *OR* e principalmente a duplicação (*fanout*) perdem informação e, portanto, não são reversíveis (NIELSEN; CHUANG, 2011). A reversibilidade pode ser observada de forma ainda mais clara em portas *multiqubits*, como a porta Toffoli (ou *CCNOT*), mostrada a seguir:

Figura 5 – Exemplo de porta Toffoli



Fonte: Criado pelo autor

Além das construções apresentadas, na computação quântica, há também alguns circuitos amplamente conhecidos por seus padrões estruturais, ou seja, são circuitos que representam classes inteiras de operações relevantes tanto do ponto de vista teórico quanto algorítmico. Entre eles, destacam-se os circuitos GHZ, QFT e QAOA, cada um associado a um tipo específico de tarefa: geração de estados fortemente emaranhados, transformação de base para algoritmos quânticos e otimização variacional, respectivamente. Esses circuitos demonstram como diferentes combinações de portas podem ser organizadas para fins específicos dentro de algoritmos quânticos.

A Tabela 2 resume as principais características desses três tipos de circuitos, destacando seus objetivos, aplicações e estruturas gerais.

Tabela 2 – Características de três classes de circuitos quânticos amplamente utilizados

Aspecto	**GHZ**	**QFT**	**QAOA**
Objetivo Geral	Geração de estados multipartite fortemente emaranhados.	Realizar a Transformada de Fourier Quântica sobre amplitudes.	Resolver problemas de otimização através de um circuito variacional parametrizado.
Tipo de Estado / Saída	Estado GHZ: $\frac{1}{\sqrt{2}}(\|00\ldots0\rangle \pm \|11\ldots1\rangle)$.	Estado no domínio da frequência ou fase, adequado para estimativas e fatoração.	Estado dependente de parâmetros ajustados por um otimizador clássico.
Estrutura Típica	Uma porta Hadamard seguida de uma cadeia de portas CNOT entre todos os qubits.	Sequência de portas Hadamard e rotações controladas (R_k), finalizada por reversão da ordem dos qubits.	Alternância entre operadores do Hamiltoniano do problema e operadores misturadores.
Portas Utilizadas	H e CNOT.	H, Controlled-R_k, SWAP (opcional).	Portas parametrizadas como $R_x(\theta)$, $R_y(\theta)$, $R_Z(\theta)$.
Aplicações Principais	Testes de coerência, validação de hardware, estudos de não-localidade.	Algoritmo de Shor, estimação de fase, simulação quântica.	Problemas de otimização combinatória (ex.: MaxCut).
Aspectos de Otimização	Simples de transpilar; útil como padrão de validação.	Pode exigir muitas portas de precisão; existe QFT aproximada.	Depende de ajustes clássicos; sensível a profundidade e ruído.
Observações	Exemplo clássico de estado altamente emaranhado.	É uma transformação unitária com eficiência superior ao caso clássico.	Importante para computadores NISQ; usado em algoritmos híbridos.

Adaptado de: Hidary (2019), Silva (2024), Younis e Iancu (2022)

2.1.6 Portas e circuitos nativos do *hardware* quântico

Na arquitetura fundamental de um computador quântico, as operações executadas correspondem diretamente às instruções fisicamente implementáveis no *hardware*, conhecidas como portas nativas (ou *basis gates*). Assim como processadores clássicos operam por meio de um conjunto reduzido de instruções de máquina, dispositivos quânticos também possuem um repertório limitado de portas que podem ser aplicadas diretamente aos *qubits* (DANGWAL et al., 2023).

A implementação dessas portas depende do tipo de tecnologia física empregada, seja por meio de pulsos de micro-ondas em *qubits* supercondutores, feixes *laser* em íons aprisionados, manipulação óptica em átomos neutros ou interferência em sistemas fotônicos

(HIDARY, 2019). Cada arquitetura define seu próprio conjunto de portas nativas, refletindo limitações físicas e características de controle específicas do dispositivo.

Quando circuitos quânticos são desenvolvidos em linguagens de alto nível ou SDKs, como qiskit, utilizam-se portas lógicas conceituais que frequentemente não correspondem diretamente às operações nativas disponíveis em um hardware específico. A ponte entre o nível abstrato e a execução física é estabelecida pelo processo de compilação (ou transpilação), no qual o compilador decompõe cada porta lógica de alto nível em sequências equivalentes de portas nativas suportadas pelo dispositivo de destino (DANGWAL et al., 2023).

Um conjunto típico de portas nativas, geralmente composto por operações de *single-qubit* combinadas com uma porta de dois-*qubits* (como a $CNOT$), pode constituir um conjunto universal de portas. Esta universalidade garante que qualquer operação quântica possa ser decomposta ou construída a partir dessa base fundamental de operações (CHEN et al., 2023).

Portanto, a diferença entre portas em código e portas nativas de *hardware* está no nível de abstração: o código usa portas lógicas conceituais (que podem ser complexas), enquanto o *hardware* executa um conjunto limitado de portas nativas elementares (implementadas por pulsos físicos). A transpilação traduz uma na outra. O ruído inerente ao *hardware* real impacta a execução dessas portas nativas, tornando a escolha eficiente da sequência de portas nativas (compilação/otimização) essencial para obter resultados confiáveis.

2.1.6.1 Ruído

Nos dispositivos quânticos da era NISQ, o termo ruído refere-se ao conjunto de imperfeições físicas e limitações operacionais que fazem com que a execução real de um circuito quântico se desvie do comportamento ideal previsto pela teoria. Em outras palavras, ruído é qualquer processo indesejado que degrada a fidelidade das operações quânticas, introduzindo erros tanto nos estados dos *qubits* quanto nas portas aplicadas (CHEN et al., 2023). Trata-se, portanto, de uma característica inerente ao *hardware* quântico contemporâneo, resultante de interações com o ambiente, instabilidade dos dispositivos e limitações nos mecanismos de controle.

Essa distinção entre o comportamento ideal e o comportamento físico fica ainda mais evidente ao considerar a diferença entre *qubits* lógicos, aqueles utilizados no nível abstrato do algoritmo, e *qubits* físicos, que correspondem aos elementos reais do dispositivo. Enquanto os qubits lógicos assumem operações perfeitas e conectividade total conforme descrito pelo circuito teórico, os qubits físicos estão sujeitos a erros, decoerência e restrições topológicas impostas pelo processador quântico real (HIDARY, 2019).

Esses efeitos ruidosos manifestam-se de diferentes formas durante a execução de circuitos quânticos. Portas de dois-*qubits*, como a $CNOT$, são particularmente propensas a erros sistemáticos devido à maior complexidade física envolvida em sua implementação. Consequentemente, suas taxas de erro são significativamente superiores às das portas de *single-qubit*, tornando-as a principal fonte de falhas nos sistemas quânticos atuais (SCHNEIDER; BURGHOLZER; WILLE, 2022).

Além disso, a arquitetura física dos dispositivos impõe restrições de conectividade, de modo que operações entre *qubits* não adjacentes exigem etapas adicionais de rote-

amento. Esse roteamento é frequentemente realizado por meio de portas *SWAP*, cada uma implementada por três portas *CNOT*. Esse processo aumenta a profundidade do circuito e, consequentemente, o acúmulo de erros, especialmente quando envolve *qubits* fisicamente distantes, nos quais a taxa de erro tende a crescer proporcionalmente à distância (DANGWAL et al., 2023). Nesse contexto, o mapeamento entre qubits lógicos e qubits físicos, conhecido como *layout* ou *qubit mapping*, torna-se uma parte importante, pois determina quais qubits do algoritmo serão atribuídos às posições físicas com menores taxas de erro e melhor conectividade.

Diante dessas limitações impostas pelo ruído e pela arquitetura física, a otimização de circuitos quânticos torna-se uma etapa essencial. O objetivo é reduzir o número de operações (principalmente *CNOTs*), minimizar a profundidade do circuito e escolher estrategicamente o mapeamento dos *qubits* lógicos nos *qubits* físicos, bem como a sequência de portas nativas, adaptando o circuito às características específicas de cada *hardware* (DANGWAL et al., 2023).

2.2 Transpilação

A transpilação quântica desempenha um papel fundamental na computação quântica prática, funcionando como um processo de tradução essencial que converte algoritmos abstratos em implementações executáveis em computadores quânticos reais.

O principal desafio da transpilação reside em adaptar as operações quânticas idealizadas às capacidades reais do *hardware*, considerando três aspectos críticos: a conectividade física entre os *qubits*, que determina quais pares podem interagir diretamente; o conjunto limitado de portas quânticas nativas que o dispositivo pode executar; e as características específicas de erro do processador, que variam entre diferentes arquiteturas (DANGWAL et al., 2023).

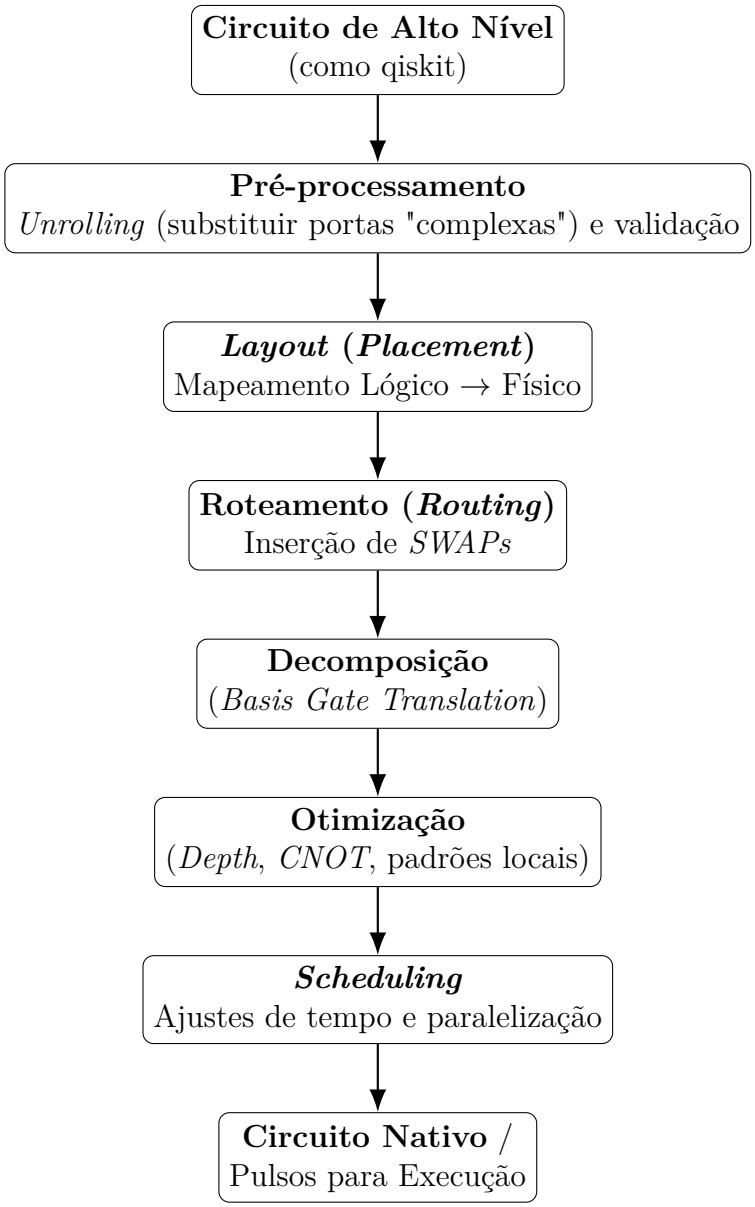
A importância da transpilação eficiente é evidente quando consideramos o equilíbrio que deve ser mantido entre a fidelidade da computação e a eficiência operacional proposta pelo algoritmo (DANGWAL et al., 2023).

2.2.1 Passes e fluxo da transpilação

O processo de transpilação é organizado como um *pipeline* de passes (*passes-based pipeline*), representado pela Figura 6. Cada passe aplica transformações locais ou globais no circuito: alguns são obrigatórios (mapeamento, roteamento) e outros são opcionais (otimizações de pós-mapeamento, inserção de pulsos de mitigação). A ordem e a combinação de passes influenciam fortemente a qualidade do executável final (DANGWAL et al., 2023).

Ou seja, a transpilação quântica se trata em converter um circuito idealizado, escrito em alto nível e sem restrições físicas, em um circuito executável compatível com o *hardware* real. Este processo envolve aplicar um *layout* que mapeia *qubits* lógicos para *qubits* físicos do processador, inserir operações de roteamento (como portas *SWAP*) quando *qubits* não conectados precisam interagir, decompor portas abstratas em portas nativas suportadas pelo dispositivo e aplicar otimizações. Por fim, existe um circuito pronto para executar respeitando tempos, paralelismo e recursos disponíveis (NASH et al., 2020).

Figura 6 – Resumo do fluxo da transpilação quântica



Fonte: Criado pelo autor

2.2.1.1 Roteamento dos *qubits*

Por causa da conectividade física limitada atual dos dispositivos quânticos atuais, o mecanismo de roteamento torna-se uma etapa necessária na transpilação, pois permite interações entre *qubits* adjancentes. Assim, quando o circuito lógico tem uma operação de dois qubits que não possuem conexão direta no *hardware*, o roteamento insere portas *SWAP*, que movem estados quânticos ao longo da topologia física até que a interação desejada seja possível. Esse processo depende de um mapeamento inicial entre *qubits* lógicos e *qubits* físicos, cuja qualidade determina diretamente o número de *SWAPs* necessários (HUA et al., 2023).

Cada porta *SWAP* é implementada por três portas *CNOT* consecutivas, o que aumenta a profundidade do circuito e a probabilidade de erro, já que portas de dois *qubits*

são ruidosas em dispositivos NISQ. Por esse motivo, minimizar a inserção de *SWAPs* é essencial para preservar a fidelidade do circuito (HUA et al., 2023).

Algoritmos modernos de roteamento procuram minimizar não apenas os *SWAPs* imediatos, mas também sua influência em possíveis otimizações subsequentes, buscando trajetórias que equilibrem custo local e impacto global no circuito (DANGWAL et al., 2023).

2.2.1.2 Decomposição de portas

A decomposição de operações quânticas transforma portas lógicas em sequências de portas nativas suportadas pelo *hardware* alvo, sendo uma etapa que possibilita a execução de algoritmos em dispositivos reais. Esse processo envolve, principalmente, reescrever portas de múltiplos *qubits* em combinações de operações de um e dois *qubits*, compatíveis com o conjunto limitado de instruções que cada processador quântico disponibiliza (KARUPPASAMY et al., 2025). Como muitos hardwares oferecem apenas rotações de um único *qubit* (como Rx, Ry, Rz) e uma única porta de dois *qubits* (como $CNOT$ ou $SWAP$), portas mais complexas como Toffoli ou Hadamard, quando não nativa precisam ser decompostas nesses blocos básicos (HUA et al., 2023).

Além de garantir compatibilidade física, a decomposição também traz uma flexibilidade no projeto de algoritmos, pois permite que pesquisadores e desenvolvedores especifiquem portas de nível mais alto sem considerar diretamente as restrições do *hardware* (HUA et al., 2023). No entanto, esse processo inevitavelmente afeta a eficiência do circuito, podendo aumentar tanto o número total de operações quanto a profundidade final, o que é impactante no desemepnho em dispositivos NISQ, onde portas adicionais introduzem mais ruído (DANGWAL et al., 2023).

2.2.1.3 Otimização e suas métricas

A otimização de circuitos quânticos tem se consolidado como uma área fundamental para o avanço da computação quântica prática, especialmente em dispositivos da era NISQ, onde limitações de ruído e conectividade impõem desafios significativos (KARUPPASAMY et al., 2025).

O desenvolvimento de técnicas de otimização permitiu melhorias cruciais no desempenho de algoritmos quânticos, abordando três objetivos principais: a minimização do número de portas quânticas, reduzindo operações desnecessárias; a redução da profundidade do circuito, aumentando a paralelização de operações; e o aumento da fidelidade das portas, garantindo maior precisão nas saídas em relação aos valores esperados (KARUPPASAMY et al., 2025).

A otimização de circuitos quânticos tem como objetivo principal aprimorar a eficiência dos circuitos quânticos que podem ser avaliadas por meio de diversas métricas como abordado por Karuppasamy et al. (2025).

A **profundidade do circuito**, medida pelo número de operações sequenciais, assume relevância no tempo de execução. Paralelamente, a **contagem total de portas** influencia diretamente a fidelidade do circuito, já que cada operação adicional representa uma potencial fonte de erro, particularmente crítica em sistemas com altas taxas de ruído.

Do mesmo modo, o **paralelismo** do circuito, capacidade de executar operações independentes simultaneamente, oferece oportunidades para acelerar a computação.

A **contagem de *qubits*** necessários reflete a eficiência na alocação de recursos em processadores quânticos atuais, onde a disponibilidade limitada de *qubits* de qualidade restringe a implementação prática de algoritmos complexos.

Dentre as operações, as **portas de dois *qubits*** consomem significativamente mais tempo de execução e apresentam taxas de erro consideravelmente maiores que portas de único *qubit*, tornando sua minimização uma prioridade nas estratégias de compilação.

A **fidelidade das portas** mede a precisão com que as portas implementadas no *hardware* quântico se aproximam do comportamento ideal. Uma maior fidelidade resulta em operações mais confiáveis e precisas.

2.3 Inteligência artificial

A Inteligência Artificial (IA), como discorrido por Vankov e Gadhoumi (2024) é a simulação de processos de inteligência humana por máquinas, especialmente sistemas computacionais, envolvendo capacidades como aprendizado (adquirir e aplicar conhecimentos a partir de dados), raciocínio (usar regras para chegar a conclusões) e autocorreção (aperfeiçoar-se com base em erros).

Aplicada desde sistemas simples até soluções complexas (como carros autônomos e assistentes virtuais), a IA tem como objetivo principal criar máquinas que operem de forma inteligente e independente, replicando habilidades humanas como visão computacional, tomada de decisão e tradução entre idiomas. Seu avanço também traz questões éticas, como impactos na privacidade e no emprego, exigindo desenvolvimento responsável. Em essência, a IA busca capacitar máquinas a executar tarefas que tradicionalmente demandariam cognição humana (VANKOV; GADHOUMI, 2024).

2.3.1 Aprendizado de máquina

O Aprendizado de Máquina (ML) é um campo da inteligência artificial que desenvolve algoritmos e modelos estatísticos capazes de executar tarefas sem serem explicitamente programados para cada ação. Ele permite que os computadores aprendam com dados, imitando a forma como humanos adquirem conhecimento, e melhorem sua precisão continuamente através da experiência (VANKOV; GADHOUMI, 2024).

Existem três principais abordagens no aprendizado de máquina:

- No **aprendizado supervisionado**, os modelos são treinados com dados rotulados, onde cada exemplo possui uma resposta correta associada, permitindo que o algoritmo aprenda a relacionar entradas com saídas esperadas.
- O **aprendizado não supervisionado** trabalha com dados sem rótulos, buscando identificar padrões ou estruturas ocultas nos dados, como agrupamentos ou relações entre variáveis.
- O **aprendizado por reforço** treina modelos através de tentativa e erro, onde o sistema recebe recompensas por ações corretas e penalizações por erros, aprendendo a tomar decisões otimizadas em um ambiente dinâmico.

A Aprendizado de Máquina se tornou fundamental por causa da sua capacidade de automatizar a capacidade de sistemas aprenderem e se adaptarem sem intervenção humana direta, assim, resolvendo problemas complexos que seriam inviáveis com programação tradicional.

2.3.1.1 Aprendizagem supervisionada

Como foi abordado por Vankov e Gadhoumi (2024), o aprendizado supervisionado é um método do Aprendizado de Máquina onde modelos são treinados com dados rotulados, ou seja, cada entrada tem sua resposta correta associada. O algoritmo aprende a mapear entradas para saídas, podendo depois fazer previsões em novos dados.

Por isso, há uma dependência da qualidade e quantidade dos dados rotulados disponíveis. Quanto mais bem detalhados forem os dados de treinamento, melhor tenderá a ser o desempenho do modelo. Sua combinação de fundamentos estatísticos com aplicações práticas faz dele uma das ferramentas mais poderosas e amplamente adotadas no ecossistema de inteligência artificial (VANKOV; GADHOUMI, 2024).

Além disso, o uso do conjunto de treinamento (*training set*) é utilizado como base para aprender os padrões, enquanto o conjunto de validação (*validation set*) serve para estimar o desempenho do modelo em dados não vistos. O fenômeno do *overfitting*, em que o modelo apresenta erro baixo no conjunto de treinamento e erro elevado no conjunto de validação, é indicativo de que o modelo aprendeu padrões específicos (ou ruídos) dos dados de treino, prejudicando sua generalização.

No treinamento supervisionado, a definição da função de perda (*loss*) é central para orientar a otimização dos parâmetros do modelo. Além da tradicional *cross-entropy*, utilizada para medir a divergência entre a distribuição prevista e a real, é possível incorporar múltiplos termos de perda como penalidades relacionadas à profundidade do modelo, regularização de *gates*, fidelidade de reconstrução, entre outros. A função de perda total pode ser expressa genericamente como uma soma ponderada de diferentes componentes:

$$L_{\text{total}} = w_{\text{ce}} L_{\text{ce}} + w_{\text{depth}} L_{\text{depth}} + w_{\text{gates}} L_{\text{gates}} + w_{\text{fidelity}} L_{\text{fidelity}} \tag{2.21}$$

onde cada termo representa um critério de otimização diferente, e os pesos w associados controlam sua influência relativa durante o treinamento do modelo (GE et al., 2024a).

2.3.1.2 Transformer

O *Transformer* é um modelo supervisionado do tipo *sequence-to-sequence*, projetado para aprender a distribuição condicional entre uma entrada e sua saída correspondente. É amplamente empregado em tarefas como tradução automática, resumo de textos e geração de linguagem natural. Sua arquitetura é composta por camadas de *encoder* e *decoder*, organizadas em blocos fundamentais que utilizam mecanismos de *self-attention* e *multi-head attention* para identificar e modelar relações contextuais relevantes entre os elementos de uma sequência (VASWANI et al., 2017).

As entradas do *Transformer*, frequentemente textos já tokenizados, são inicialmente convertidas em *embeddings*, vetores densos aprendidos contendo informações semânticas e sintáticas de cada *token*. Como a arquitetura não possui elementos recorrentes ou convolucionais, é necessário incorporar codificações posicionais a esses *embeddings*, garantindo sensibilidade à ordem dos elementos na sequência. Essas codificações podem ser calculadas

por funções determinísticas, como seno e cosseno, ou aprendidas durante o treinamento (CHEOK; ZHANG, 2023).

No bloco do *encoder*, cada camada realiza inicialmente a operação de *multi-head attention*. Nesse processo, cada posição da entrada é capaz de ponderar e integrar informações de todas as demais posições, com pesos determinados por similaridades aprendidas, promovendo uma composição rica e global do contexto contextual. Para viabilizar esse mecanismo, o modelo realiza projeções lineares dos vetores de entrada (*embeddings*), convertendo-os em representações intermediárias denominadas *queries*, *keys* e *values*. Um fator de escalonamento é aplicado durante o cálculo da atenção para conferir maior estabilidade ao processo de treinamento e melhorar a robustez dos gradientes (RODRIGUES; BRANCO, 2024).

Após a etapa de *self-attention*, a representação de cada *token* passa por uma rede *feed-forward* aplicada individualmente a cada posição. Essa técnicas são conectadas através de atalhos residuais e por normalização em cada camada (*layer normalization*), pois são essenciais para mitigar problemas de degradação dos gradientes e estabilizar o treinamento (VASWANI et al., 2017).

O bloco de *decoder* apresenta algumas particularidades em relação ao *encoder*. Primeiramente, utiliza a *self-attention* com máscara para garantir que, durante a geração, cada *token* só tenha acesso a posições anteriores ou atuais. Além disso, inclui uma etapa de *cross-attention*, na qual o *decoder* traz às representações produzidas pelo *encoder*, permitindo alinhamento eficaz entre entrada e saída. Ao final, as representações geradas pelo *decoder* são projetadas por meio de uma camada linear e submetidas a uma função *softmax*, resultando em uma distribuição de probabilidades sobre os possíveis *tokens* de saída (ZAIB et al., 2021).

Além disso, a técnica de *teacher forcing* também é frequentemente empregada, fornecendo ao *decoder* o prefixo verdadeiro da sequência-alvo em cada etapa de treinamento, o que contribui para acelerar e estabilizar a convergência do processo de aprendizagem (ZAIB et al., 2021).

Outro aspecto relevante é o chamado *depth improvement*, conceito diretamente associado ao número de camadas empilhadas de *encoder* e *decoder* no modelo. O aumento da profundidade permite ao *Transformer* representar funções cada vez mais complexas e hierárquicas, potencializando a aprendizagem de relações de alto nível nas sequências de entrada e saída (VASWANI et al., 2017).

2.4 Ferramentas

2.4.1 Pytorch

O treinamento de inteligência artificial requer uma combinação refinada de técnicas computacionais para garantir que o modelo aprenda de forma estável, generalize bem e evite problemas comuns como sobreajuste ou instabilidades numéricas. *PyTorch* é um *framework* de *deep learning*, criado pelo *Facebook AI Research* em 2016, com componentes disponíveis de forma nativa que ajudam nesse treinamento (RAYARAO; DONIKENA, 2025).

O `AdamW` é um otimizador avançado utilizado para atualizar os parâmetros de uma

rede neural durante o treino. Ele é um dos otimizadores adaptativos, ajusta a velocidade (taxa de aprendizado) de cada parâmetro automaticamente, o que costuma acelerar e estabilizar o processo de aprendizagem (WRIGHT; DEMEURE, 2021).

```
import torch.optim as optim

optimizer = optim.AdamW(
    model.parameters(),     # Parâmetros do modelo a serem otimizados
    lr=1e-4,                # Taxa de aprendizado inicial
    weight_decay=1e-5       # Força da regularização
)
```

Durante o treinamento, pode ocorrer de a função de perda (*loss*) ou outra métrica de validação estagnar, mesmo que ainda seja possível melhorar o modelo ajustando a taxa de aprendizado (*learning rate*). O `ReduceLROnPlateau` é um *scheduler* que monitora uma métrica do treinamento, como o valor de *loss* em validação, e reduz automaticamente o *learning rate* quando detecta falta de progresso. Isso é fundamental para o treinamento em estágios finais ou escapar de mínimos locais (RAYARAO; DONIKENA, 2025).

```
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,          # Otimizador sendo usado
    mode='min',         # Modo 'min': busca reduzir a métrica (loss)
    factor=0.5,         # Multiplica o learning rate por 0.5 ao ativar
    patience=5          # Espera 5 épocas antes de reduzir o learning rate
)

# O scheduler deve ser chamado regularmente informando a métrica monitorada:
scheduler.step(val_loss)
```

Exploding gradients ocorre quando, durante o treino, os gradientes calculados se tornam excessivamente grandes, o que gerar saltos abruptos na atualização dos pesos ou até valores infinitos. O `gradient clipping` limita o valor máximo que a norma dos gradientes pode atingir ao longo das atualizações. Dessa forma, garante-se que as mudanças nos parâmetros sejam graduais e estáveis, facilitando a convergência do modelo (JIANG et al.,).

```
torch.nn.utils.clip_grad_norm_(
    model.parameters(),    # Parâmetros do modelo
    max_norm=1.0          # Limite máximo para a norma dos gradientes
)
```

O `Dropout` é uma técnica de regularização simples que durante o treinamento, forçando o treinamento não depender excessivamente de caminhos ou combinações específicas e promovendo redundância saudável nas representações aprendidas (RAYARAO; DONIKENA, 2025).

```
import torch.nn as nn

dropout = nn.Dropout(p=0.1)  # 10% das ativações zeradas aleatoriamente
x = dropout(x)               # Aplica dropout durante o treino
```

A normalização de ativações é uma outra estratégia para estabilizar e acelerar o aprendizado. O `LayerNorm` normaliza as ativações a cada amostra, ao longo dos recursos, subtraindo a média e dividindo pelo desvio-padrão. Isso garante que as ativações fiquem em uma escala padronizada (ZHUANG; LIU; CUTKOSKY, 2022).

```
layer_norm = nn.LayerNorm(normalized_shape)
y = layer_norm(x)  # x é normalizado por amostra
```

2.4.2 Qiskit

Desenvolvido pela IBM, o qiskit se estabeleceu como uma das principais bibliotecas para programação de computadores quânticos, oferecendo um kit de desenvolvimento de software (SDK) baseado em *python* abrangente para criação de circuitos, transpilação, otimização, simulação e execução em *hardware* real de algoritmos quânticos. Sua arquitetura permite a adaptação flexível de programas quânticos às especificações de diferentes processadores, considerando suas restrições físicas e capacidades operacionais (HIDARY, 2019).

Dentre os módulos existentes, destacamos o `Qiskit Aer` para simulação com suporte a ruído customizável e métodos de simulação para circuitos com até cerca de 50 *qubits*, além de ferramentas para otimização e análise de resultados (IBM Quantum Documentation, 2023).

Um exemplo prático de simulação local pode ser observado a seguir:

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator

qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)
qc.measure_all()

simulator = AerSimulator()
job = simulator.run(qc, shots=1000)
result = job.result()
counts = result.get_counts()

print("Resultados da simulação local:", counts)
```

2.4.3 IBM Quantum

O ecossistema *IBM Quantum* disponibiliza o acesso a uma crescente rede de computadores quânticos de uso remoto por meio de serviços em nuvem, estando plenamente integrado ao *qiskit* por meio do *IBM Quantum Runtime*, com o limitador de 10 minutos de compilção mensal. O `Runtime`, módulo do *qiskit* centraliza o gerenciamento de recursos, a execução otimizadora de circuitos, a mitigação dinâmica de erros (incluindo calibração automática, modelagem de ruído, e técnicas para minimizar o impacto de fidelidade limitada dos qubits) e o gerenciamento de filas de jobs, tudo com interface homogênea e programável (IBM Quantum Documentation, 2023).

Exemplo de processo de submissão:

```python
from qiskit import QuantumCircuit, transpile
from qiskit_ibm_runtime import QiskitRuntimeService, SamplerV2 as Sampler

qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)
qc.measure_all()

service = QiskitRuntimeService()
backend = service.least_busy(
    operational=True,
    simulator=False,
    min_num_qubits=2
)
transpiled_qc = transpile(qc, backend=backend)

sampler = Sampler(backend)
job = sampler.run([transpiled_qc], shots=1024)
result = job.result()
counts = result[0].data.meas.get_counts()

print("Resultados do hardware real IBM:", counts)
```

2.4.4 MQT Bench

O MQT *Bench* é uma suíte de *benchmarks* do *Munich Quantum Toolkit* desenvolvida para avaliar de ferramentas e fluxos de automação na computação quântica. Disponibiliza um conjunto de circuitos padronizados, variando desde experimentos com poucos até centenas de *qubits*, totalizando dezenas de milhares de instâncias. Essa diversidade permite obter resultados comparáveis, reprodutíveis e generalizáveis, tanto para análise acadêmica quanto para avaliação prática de compiladores e arquiteturas quânticas. Os circuitos do MQT *Bench* podem ser acessados via *web* ou diretamente por um pacote *python*, facilitando sua integração em experimentos automatizados (QUETSCHLICH; BURGHOLZER; WILLE, 2023).

A suíte opera em quatro níveis de abstração (*benchmark levels*), representando diferentes estágios do fluxo de compilação: ***algorithmic***, circuitos ideais livres de restrições

de hardware; ***target-independent***, adaptados para um conjunto universal de portas, porém ainda independentes do dispositivo; ***native gates***, traduzidos para um conjunto nativo de portas de um alvo específico; e ***mapped***, circuitos totalmente mapeados com alocação física de qubits e roteamento considerando a conectividade real do dispositivo (QUETSCHLICH; BURGHOLZER; WILLE, 2023).

O MQT Bench suporta algoritmos clássicos (como Grover e GHZ), algoritmos variacionais (como QAOA), circuitos aleatórios e aplicações domínios como química e finanças. Os resultados podem ser exportados em OpenQASM 2.0, OpenQASM 3.0 (padrão aberto, textual, utilizado para descrever circuitos quânticos), e QPY (formato binário do Qiskit para serializar circuitos *python*), cada um incluindo um cabeçalho padronizado com metadados relevantes: data de geração, níveis de abstração, conjunto de portas utilizadas, dispositivo alvo, parâmetros de compilação e informações do ambiente. Os arquivos seguem convenção de nomenclatura detalhada, como `qft_mapped_qiskit_7.qasm`, codificando algoritmo, nível, alvo e o número de qubits.

A geração de circuitos pode ser feita tanto pela **CLI** quanto pela API Python, através de funções como `get_benchmark` (para construção de circuitos parametrizados) e `save_circuit` (para exportação em diferentes formatos e diretórios). Por exemplo, para gerar em Python um circuito QFT mapeado para o dispositivo *ionq_forte* com 7 qubits e otimização 2 (WILLE et al., 2024):

```python
from mqt.bench import BenchmarkLevel, get_benchmark
from mqt.bench.output import save_circuit, OutputFormat

# Gera um circuito QFT mapeado para IonQ Forte e salva em OpenQASM 3
qc = get_benchmark(
    benchmark="qft",
    level=BenchmarkLevel.MAPPED,
    circuit_size=7,
    target="ionq_forte",
    opt_level=2
)

save_circuit(
    qc,
    filename="qft_mapped_ionq_forte_opt2_7",
    level=BenchmarkLevel.MAPPED,
    output_format=OutputFormat.QASM3,
    target="ionq_forte"
)
```

2.5 Trabalhos relacionados

O artigo proposto pelo Kirilov e Shevchenko (2024) apresenta uma análise abrangente sobre métodos de otimização da profundidade de circuitos quânticos, abordando desde fundamentos teóricos até técnicas avançadas de compilação, com o objetivo de melhorar a eficiência computacional e enfrentar os desafios de escalabilidade em dispositivos quânticos ruidosos.

O autor destaca a importância de desenvolver métodos independentes de *hardware*, capazes de adaptar-se a diferentes arquiteturas, e enfatiza a necessidade de validação experimental robusta para assegurar a aplicabilidade prática das otimizações propostas.

Uma das contribuições centrais do trabalho reside na discussão sobre técnicas de síntese exata, no contexto do número especificamente de portas T, operações custosas em termos de recursos. Outro aspecto relevante é a abordagem que integra otimizações em múltiplos níveis, desde o *hardware* até o compilador, permitindo uma análise mais coerente das compensações entre profundidade do circuito, fidelidade de operação e restrições de arquitetura. O Kirilov e Shevchenko (2024) ressalta que essa estratégia é particularmente útil em dispositivos NISQ, onde a minimização do acúmulo de erros é crucial.

Além disso, o artigo dedica espaço à discussão sobre a necessidade de avaliações experimentais amplas, sugerindo *benchmarks* padronizados como mecanismo para comparar avanços. Propõe também a incorporação de métodos de aprendizado de máquina para prever resultados de otimização, antecipando tendências de pesquisa que, como visto em trabalhos mais recentes, já se concretizam na adoção de algoritmos adaptativos ou autoajustáveis para o fluxo de compilação.

Outro trabalho é do Quetschlich, Burgholzer e Wille (2025), onde é proposto uma abordagem para automatizar tanto a seleção do dispositivo quântico quanto o processo de compilação associado, com o objetivo de evitar a escolha manual e custosa de configurações. O *framework* combina técnicas de aprendizado por reforço em uma estrutura integrada capaz de recomendar o *hardware* mais adequado e, em seguida, compilar o circuito especificamente para esse dispositivo.

Na etapa de otimização da compilação, o *MQT Predictor* modela o fluxo de transpilação como um Processo de Decisão de Markov, no qual cada passe de compilação é tratado como uma ação. Um agente de RL aprende sequências eficientes de passes a partir de centenas de circuitos do *MQT Bench*, considerando métricas como fidelidade esperada ou profundidade crítica. Para permitir a combinação de passes de diferentes SDKs e reduzir dependência de fornecedor.

Nesse contexto, vale detalhar que um Processo de Decisão de Markov (Markov Decision Process) é um modelo matemático amplamente utilizado para descrever problemas de tomada de decisão sequencial, em que um agente realiza ações em ambientes que evoluem ao longo do tempo. No *MQT Predictor*, cada estado corresponde à configuração atual do circuito (por exemplo, quais portas já foram adaptadas, quais restrições de hardware já foram satisfeitas), e cada ação corresponde à aplicação de um passo de compilação específico. As decisões são tomadas visando maximizar uma recompensa associada à métrica escolhida (fidelidade, profundidade, etc.) e, por ser um processo de Markov, é considerado que a escolha da próxima ação depende apenas do estado atual do circuito, facilitando a aprendizagem de estratégias eficientes e generalizáveis para diferentes arquiteturas.

A segunda etapa aborda a seleção automática do dispositivo, tratada como um problema de classificação supervisionada. O modelo recebe características estruturais do circuito, como profundidade, número de *qubits* e métricas derivadas de paralelismo e comunicação, e detecta qual dispositivo, entre diversas arquiteturas de *qubits* supercondutores e íons aprisionados, deverá oferecer o melhor desempenho para a figura de mérito escolhida.

3 Desenvolvimento

Este capítulo apresenta o desenvolvimento completo do sistema proposto para análise da otimização de circuitos quânticos utilizando modelos baseados em arquitetura Transformer. São descritas a metodologia adotada, o conjunto de dados utilizado, a evolução do modelo, bem como os procedimentos de teste e os resultados obtidos.

3.1 Metodologia

A metodologia deste trabalho baseia-se na aplicação de aprendizado supervisionado para investigar com modelos *Transformer* capturando padrões de otimização utilizados por transpiladores quânticos. Este projeto utiliza pares de circuitos (original, otimizado) como dados de treinamento, permitindo análise explícita dos pesos atribuídos a diferentes métricas de otimização.

Tendo como objetivo compreender como a otimização acontece e revelar a importância relativa de fatores como profundidade, número de portas, fidelidade e portas críticas (*CNOT*, *multi-qubit* e *SWAP*).

A arquitetura adotada é um *Transformer* do tipo *encoder-decoder*, implementado em `PyTorch`. No *encoder*, o circuito original é processado por camadas de *self-attention*, permitindo que o modelo aprenda interações entre portas e dependências entre qubits. O mecanismo de *self-attention* calcula pesos que determinam quais partes do circuito são relevantes para cada decisão. O *decoder* gera o circuito otimizado token por token, utilizando *self-attention* para manter coerência com o que já foi gerado e *cross-attention* para focar em regiões relevantes do circuito original.

O modelo utiliza dimensão de 512 para representação de *tokens*, 8 *multi-head attention* paralelas (cada uma com 64 dimensões), 6 camadas tanto no *encoder* quanto no *decoder*, dimensão *feedforward* de 2048, comprimento máximo de 500 *tokens* e *dropout* de 0.1.

O processo foi estruturado em cinco versões iterativas (v1.0 a v5.0), onde cada versão foi ajustada a partir da análise dos resultados da versão anterior, permitindo investigar experimentalmente o impacto de diferentes configurações da função de perda no desempenho do modelo.

3.2 Dados Utilizados

O conjunto de dados foi construído a partir de circuitos quânticos provenientes do *MQT Bench*. Foram incluídos três tipos de circuitos: GHZ que cria estados de emaranhamento máximo; QFT fundamental para algoritmos como o de Shor; e QAOA para problemas de otimização combinatória.

Para cada tipo de circuito, foram geradas versões variando de 3 a 10 *qubits*. A geração dos pares de treinamento segue três etapas: primeiro, o circuito é obtido do *MQT Bench* no nível `INDEP` (*target-independent*), que representa circuito em sua forma algorítmica sem otimizações de *hardware*; segundo, o circuito é submetido ao transpilador

qiskit com nível de otimização 3, que aplica decomposição de portas, cancelamento de portas adjacentes, comutação e síntese de blocos; terceiro, o circuito original e sua versão otimizada formam o par de treinamento.

```
from mqt.bench import get_benchmark, BenchmarkLevel
from qiskit import transpile
from qiskit_aer import AerSimulator

benchmarks = ["ghz", "qft", "qaoa"]
sizes = [3, 4, 5, 6, 7, 8, 10]

for bench in benchmarks:
    for size in sizes:
        # Circuito original do MQT Bench (nivel independente)
        qc = get_benchmark(bench, BenchmarkLevel.INDEP, size)
        original_circuits.append(qc)

        # Otimizar com Qiskit O3
        backend = AerSimulator()
        opt_qc = transpile(qc, backend=backend, optimization_level=3)
        optimized_circuits.append(opt_qc)
```

Para cada rodada de treinamento, são gerados até 50 circuitos, divididos em 80% para treinamento e 20% para validação. Os testes em *hardware* real utilizam um conjunto independente de 12 circuitos (GHZ, QFT e QAOA com 3, 4, 5 e 6 qubits), completamente separado do conjunto de treinamento para avaliar a capacidade de generalização.

```
def create_ghz_circuit(num_qubits: int) -> QuantumCircuit:
    qc = QuantumCircuit(num_qubits)
    qc.h(0)  # Hadamard: |0> -> |+>
    for i in range(num_qubits - 1):
        qc.cx(i, i+1)  # CNOT em cadeia
    return qc

def create_qft_circuit(num_qubits: int) -> QuantumCircuit:
    from qiskit.circuit.library import QFT
    return QFT(num_qubits=num_qubits, do_swaps=True)

def create_qaoa_circuit(num_qubits: int) -> QuantumCircuit:
    qc = QuantumCircuit(num_qubits)
    for i in range(num_qubits):
        qc.h(i)  # Camada de mistura
    for i in range(num_qubits - 1):
        qc.cx(i, i+1)
        qc.rz(np.pi/4, i+1)
        qc.cx(i, i+1)  # Camada de interacao
    return qc
```

3.3 Processo de Treinamento

O processo de treinamento envolve a conversão de circuitos quânticos em sequências numéricas (tokenização), a configuração do otimizador e função de perda, e o monitoramento do progresso através de *logs*.

A tokenização é o processo de converter um circuito quântico (estrutura de dados complexa com portas, *qubits* e parâmetros) em uma sequência de números inteiros que o modelo *Transformer* pode processar. Esse processo é análogo à tokenização de texto em processamento de linguagem natural, onde palavras são convertidas em índices de um vocabulário fixo.

O vocabulário do tokenizador é composto por 113 *tokens* divididos em quatro categorias. A primeira categoria são os tokens especiais (5 *tokens*) que controlam o fluxo da sequência: `[PAD]` (índice 0) é usado para completar sequências menores que o comprimento máximo, garantindo que todos os *batches* tenham o mesmo tamanho; `[UNK]` (índice 1) representa portas ou parâmetros não reconhecidos pelo vocabulário; `[START]` (índice 2) marca o início de uma sequência de circuito, sinalizando ao decoder que deve começar a gerar; `[END]` (índice 3) marca o fim da sequência, indicando que o circuito está completo; e `[BARRIER]` (índice 4) representa barreiras de sincronização que impedem reordenação de portas pelo compilador.

A segunda categoria são os tokens de portas (32 tokens) que representam as operações quânticas suportadas:

- **Portas de um qubit:**
  - `GATE_id` (identidade)
  - `GATE_x`, `GATE_y`, `GATE_z` (portas de Pauli)
  - `GATE_h` (Hadamard)
  - `GATE_s`, `GATE_t` (portas de fase)
  - `GATE_sx` (raiz de X)
- **Portas de dois qubits:**
  - `GATE_cx` (CNOT)
  - `GATE_cy`, `GATE_cz` (controladas)
  - `GATE_ch` (Hadamard controlado)
  - `GATE_swap` (troca de qubits)
- **Portas de três qubits:**
  - `GATE_ccx` (Toffoli)
  - `GATE_cswap` (Fredkin)
- **Portas parametrizadas:**
  - `GATE_rx`, `GATE_ry`, `GATE_rz` (rotações)
  - `GATE_u1`, `GATE_u2`, `GATE_u3` (portas universais)

A terceira categoria são os *tokens* de *qubits* (128 *tokens*) que identificam em qual *qubit* físico a operação é aplicada, de `QB_0` a `QB_127`. Por exemplo, se uma porta Hadamard é aplicada no *qubit* 3, ela será representada pela sequência [`GATE_h`, `QB_3`]. Para portas de dois *qubits* como *CNOT*, são necessários dois *tokens* de *qubit*: [`GATE_cx`, `QB_0`, `QB_1`] representa um *CNOT* com controle no *qubit* 0 e alvo no *qubit* 1.

A quarta categoria são os *tokens* de parâmetros (64 *tokens*) que representam ângulos de rotação discretizados. Como os parâmetros de portas como RZ são valores contínuos no intervalo $[-\pi, \pi]$, eles precisam ser convertidos para tokens discretos. O intervalo é dividido em 64 bins de aproximadamente $\frac{2\pi}{64} \approx 0.098$ radianos cada. O *token* `PARAM_0` representa ângulos próximos de $-\pi$, `PARAM_32` representa ângulos próximos de 0, e `PARAM_63` representa ângulos próximos de π. A discretização usa a função `arctan2(sin(`θ`), cos(`θ`))` para normalizar o ângulo, seguida de `np.digitize` para mapear ao bin correspondente.

O processo de tokenização percorre cada instrução do circuito em ordem: inicia com o token START, para cada instrução converte o nome da porta para `GATE_nome`, converte cada qubit envolvido para `QB_índice`, discretiza parâmetros contínuos em tokens `PARAM_bin`, e finaliza com o token END.

```python
def tokenize_circuit(self, circuit: QuantumCircuit) -> List[int]:
    tokens = [self.start_id]  # Iniciar com START

    for instruction in circuit.data:
        operation = instruction.operation
        qubits = instruction.qubits
        gate_name = operation.name.lower()

        # Converter porta para token
        gate_token = f"GATE_{gate_name}"
        tokens.append(self.token_to_id.get(gate_token, self.unk_id))

        # Converter cada qubit para token
        for qubit in qubits:
            qubit_idx = circuit.find_bit(qubit).index
            tokens.append(self.token_to_id[f"QB_{qubit_idx}"])

        # Discretizar parametros
        if hasattr(operation, 'params') and operation.params:
            for param in operation.params:
                if isinstance(param, (int, float)):
                    param_norm = np.arctan2(np.sin(param), np.cos(param))
                    bins = np.linspace(-np.pi, np.pi, 64)
                    bin_idx = np.clip(np.digitize(param_norm, bins) - 1, 0, 63)
                    tokens.append(self.token_to_id[f"PARAM_{bin_idx}"])

    tokens.append(self.end_id)  # Finalizar com END
    return tokens
```

Para ilustrar o processo, considere um circuito GHZ de 3 *qubits*, que contém uma porta Hadamard no qubit 0 seguida de dois *CNOTs* em cadeia.

A tokenização deste circuito produz a sequência: primeiro o *token* `[START]` (índice 2) indica início do circuito; depois a porta Hadamard é convertida para `GATE_h` seguido de `QB_0` indicando que é aplicada no *qubit* 0; o primeiro *CNOT* vira `GATE_cx` seguido de `QB_0` (controle) e `QB_1` (alvo); o segundo *CNOT* vira `GATE_cx` seguido de `QB_1` (controle) e `QB_2` (alvo); finalmente `[END]` (índice 3) marca o fim. A sequência final de índices seria aproximadamente: [2, 9, 37, 22, 37, 38, 22, 38, 39, 3], onde cada número corresponde ao índice do *token* no vocabulário.

O processo inverso, chamado detokenização, reconstrói um circuito Qiskit a partir de uma sequência de tokens gerada pelo modelo. O detokenizador percorre os tokens ignorando `[START]`, `[END]` e `[PAD]`, identifica cada token de porta e coleta os tokens de qubit e parâmetro subsequentes conforme a aridade da porta (1 qubit para H, 2 para CNOT, etc.), reconstrói os parâmetros contínuos a partir dos bins discretizados (multiplicando o índice do bin pelo tamanho do bin e subtraindo π), e adiciona cada operação ao circuito Qiskit resultante.

O otimizador utilizado foi o `AdamW` que mantém médias móveis do gradiente (primeiro momento) e do quadrado do gradiente (segundo momento), adaptando a taxa de aprendizado individualmente para cada um dos parâmetros. A taxa de aprendizado inicial foi definida como 1×10^{-4}, valor típico para *Transformers*, e o *weight decay* como 1×10^{-5}, que adiciona uma pequena penalidade à magnitude dos pesos para evitar *overfitting*.

O *scheduler* `ReduceLROnPlateau` monitora a perda de validação e, quando ela deixa de melhorar por 5 épocas consecutivas, reduz a taxa de aprendizado multiplicando por 0.5. Essa estratégia permite que o modelo faça ajustes grosseiros no início do treinamento (com taxa alta) e ajustes finos quando se aproxima de um mínimo local (com taxa baixa).

O treinamento incorpora três técnicas de estabilização. O *gradient clipping* limita a norma máxima do vetor de gradientes a 1.0, prevenindo gradientes explosivos que são comuns em *Transformers* devido às multiplicações em cascata nas camadas. Quando a norma do gradiente excede 1.0, todos os componentes são escalados proporcionalmente. *Layer Normalization* é aplicada após cada subcamada de atenção e *feedforward* dentro do *Transformer*, normalizando as ativações para ter média zero e variância unitária. *Dropout* com taxa de 0.1 desativa aleatoriamente 10% dos neurônios durante o treinamento, forçando o modelo a não depender excessivamente de nenhum neurônio específico.

O *early stopping* interrompe o treinamento automaticamente quando a perda de validação deixa de apresentar progresso significativo. A `patiente` (tempo de espera) foi configurada entre 15 e 30 épocas dependendo da versão, e a melhoria mínima (`min_delta`) como 0.001, ou seja, uma redução na perda menor que 0.001 não é considerada melhoria real. O tamanho dos *batches* foi mantido reduzido (2 a 4 circuitos por iteração) devido ao custo computacional elevado da avaliação de métricas durante o treinamento.

```
# Otimizador AdamW
self.optimizer = optim.AdamW(
    self.model.parameters(),
    lr=1e-4,
    weight_decay=1e-5
)

# Scheduler: reduz LR quando val_loss estagna
```

```
self.scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    self.optimizer, mode='min', factor=0.5, patience=5
)

# Durante o treinamento:
self.optimizer.zero_grad()
loss.backward()
torch.nn.utils.clip_grad_norm_(self.model.parameters(), max_norm=1.0)
self.optimizer.step()
```

A função de perda passou por uma evolução significativa ao longo das versões, refletindo um processo incremental de aprendizado sobre quais métricas são mais relevantes para otimização de circuitos quânticos.

A versão 1.0 utilizava exclusivamente *cross-entropy*, a função de perda padrão para problemas de classificação de sequências. A *cross-entropy* mede quão bem as probabilidades previstas pelo modelo correspondem aos *tokens* corretos do circuito otimizado alvo.

A versão 2.0 introduziu uma função de perda multi-objetivo, adicionando componentes que penalizam ou recompensam o modelo com base em métricas do circuito gerado. Cada componente calcula a melhoria relativa entre o circuito original e o circuito *target* (otimizado pelo *qiskit*): $L_{\text{total}} = w_{\text{ce}}L_{\text{ce}} + w_{\text{depth}}L_{\text{depth}} + w_{\text{gates}}L_{\text{gates}} + w_{\text{fidelity}}L_{\text{fidelity}} + w_{\text{cx}}L_{\text{cx}}$.

A versão 3.0 triplicou o peso de profundidade (w_{depth}: $0.5 \rightarrow 1.5$) para corrigir aumentos de profundidade observados nos testes da v2.0. A versão 4.0 adicionou componentes para portas *multi-qubit* e *SWAP*, após identificar que essas portas são particularmente críticas em dispositivos NISQ. A versão 5.0 simplificou a função removendo profundidade, número de portas e fidelidade, focando exclusivamente nas portas problemáticas (*CNOT*, *multi-qubit*, *SWAP*), com maior peso atribuído a *SWAP* (1.5) por ser a mais custosa. A evolução dos pesos pode ser vista em Tabela 3.

Tabela 3 – Evolução dos pesos da função de perda entre versões

Versão	w_{ce}	w_{depth}	w_{gates}	$w_{fidelity}$	w_{cx}	w_{multi}	w_{swap}
v1.0	1.0	—	—	—	—	—	—
v2.0	1.0	0.5	0.3	0.2	0.4	—	—
v3.0	1.0	1.5	0.2	0.2	0.4	—	—
v4.0	1.0	1.5	0.2	0.2	0.4	0.5	0.6
v5.0	1.0	—	—	—	1.0	1.2	1.5

```
def _calculate_multi_objective_loss(self, original_circuit, target_circuit):
    orig = self.metrics_calculator.calculate_all_metrics(original_circuit)
    target = self.metrics_calculator.calculate_all_metrics(target_circuit)

    # Componente CX: reducao de portas CNOT
    if orig["cx_gates"] > 0:
        cx_loss = -(orig["cx_gates"] - target["cx_gates"]) / orig["cx_gates"]
    else:
        cx_loss = target["cx_gates"] * 0.1
```

```
    # Componente multi-qubit
    if orig["multi_qubit_gates"] > 0:
        multi_loss = -(orig["multi_qubit_gates"] - target["multi_qubit_gates"])
                     / orig["multi_qubit_gates"]
    else:
        multi_loss = target["multi_qubit_gates"] * 0.15

    # Componente SWAP (equivalente a 3 CNOTs)
    if orig["swap_gates"] > 0:
        swap_loss = -(orig["swap_gates"] - target["swap_gates"])
        / orig["swap_gates"]
    else:
        swap_loss = target["swap_gates"] * 0.2

    return self.weight_cx * cx_loss + self.weight_multi_qubit * multi_loss \
           + self.weight_swap * swap_loss
```

O sistema de *logs* registra a cada época: perda de treinamento, perda de validação, taxa de equivalência lógica e taxa de aprendizado. São salvos automaticamente o melhor modelo (`best_model.pt`), modelo sem *overfitting* (`best_no_overfitting_model.pt`) e *checkpoints* por época.

3.4 Processo de Testes

Os testes foram executados no *backend* IBM `ibm_fez` através do `Qiskit Runtime Service`, avaliando as versões v1.0 a v5.0 em 12 circuitos (GHZ, QFT e QAOA com 3, 4, 5 e 6 *qubits*). O fluxo de teste para cada circuito consiste em: gerar o circuito original; otimizar com *Transformer* (tokenizar, passar pelo *encoder-decoder*, detokenizar); otimizar com *qiskit* `O3`; transpilar ambos para o *backend* IBM; executar com 1024 *shots*; e coletar métricas.

```
# Tokenizar circuito original
tokens = self.tokenizer.tokenize_circuit(circuit)
src_tensor = torch.tensor([tokens], dtype=torch.long, device=self.device)

# Gerar circuito otimizado via decoder
generated_tokens = self.model.generate(
    src=src_tensor,
    max_length=min(len(tokens) * 2, 1000),
    temperature=0.8,
    top_k=50,
    start_id=self.tokenizer.start_id,
    end_id=self.tokenizer.end_id
)

# Detokenizar para circuito Qiskit
optimized = self.tokenizer.detokenize_circuit(
    generated_tokens[0].cpu().tolist(), num_qubits
```

```
)
```

As métricas avaliadas são: profundidade (número de camadas), total de portas, portas *CNOT*, portas *multi-qubit*, portas *SWAP*, fidelidade esperada, tempo de execução e ruído real (calculado como $1 - P(\text{estado esperado})$).

A seguir são apresentados os dados brutos dos testes de cada versão.

Tabela 4 – Transformer vs Qiskit (Version 1.0)

Circuit	**Depth (T)**	**Depth (Q)**	**Gates (T)**	**Gates (Q)**	**Fidelity (T)**	**Winner**
ghz_3	9	9	15	15	0.860	Tie
ghz_4	12	12	21	21	0.810	Tie
ghz_5	15	15	27	27	0.762	Tie
ghz_6	18	18	33	33	0.718	Tie
qft_3	35	35	46	46	0.630	Tie
qft_4	55	59	82	82	0.439	Transformer
qft_5	100	100	137	137	0.252	Tie
qft_6	151	120	231	231	0.098	Qiskit
qaoa_3	19	19	25	25	0.778	Tie
qaoa_4	27	27	33	33	0.718	Tie
qaoa_5	36	36	41	41	0.662	Tie
qaoa_6	43	43	49	49	0.611	Tie
Summary: Transformer wins: 1, Qiskit wins: 1, Ties: 10, Avg. depth improvement: -225.0%						

Tabela 5 – Transformer vs Qiskit (Version 2.0)

Circuit	**Depth (T)**	**Depth (Q)**	**Gates (T)**	**Gates (Q)**	**Fidelity (T)**	**Winner**
ghz_3	3	9	9	15	0.914	Transformer
ghz_4	11	12	14	21	0.869	Transformer
ghz_5	3	15	6	27	0.941	Transformer
ghz_6	19	18	31	33	0.732	Tie
qft_3	32	36	44	46	0.643	Transformer
qft_4	3	55	12	84	0.886	Transformer
qft_5	3	81	6	137	0.941	Transformer
qft_6	15	113	21	217	0.810	Transformer
qaoa_3	19	19	27	27	0.762	Tie
qaoa_4	27	27	35	35	0.703	Tie
qaoa_5	36	36	43	43	0.649	Tie
qaoa_6	44	44	51	51	0.599	Tie
Summary: Transformer wins: 7, Qiskit wins: 0, Ties: 5, Avg. depth improvement: +1970.7%						

Tabela 6 – Transformer vs Qiskit (Version 3.0)

Circuit	**Depth (T)**	**Depth (Q)**	**Gates (T)**	**Gates (Q)**	**Fidelity (T)**	**Winner**
ghz_3	9	9	15	15	0.860	Tie
ghz_4	11	12	14	21	0.869	Transformer
ghz_5	15	15	21	27	0.810	Transformer
ghz_6	19	18	28	33	0.755	Tie
qft_3	3	32	9	44	0.914	Transformer
qft_4	68	53	82	82	0.439	Qiskit
qft_5	100	79	137	135	0.252	Qiskit
qft_6	150	144	217	218	0.113	Tie
qaoa_3	19	19	27	27	0.762	Tie
qaoa_4	27	27	35	35	0.703	Tie
qaoa_5	36	36	43	43	0.649	Tie
qaoa_6	43	43	51	51	0.599	Tie
Summary: Transformer wins: 3, Qiskit wins: 2, Ties: 7, Avg. depth improvement: -107.6%						

Tabela 7 – Transformer vs Qiskit (Version 4.0)

Circuit	**Depth (T)**	**Depth (Q)**	**Gates (T)**	**Gates (Q)**	**Fidelity (T)**	**Winner**
ghz_3	9	9	15	15	0.860	Tie
ghz_4	3	12	12	21	0.886	Tie
ghz_5	3	15	15	27	0.860	Tie
ghz_6	18	18	33	33	0.718	Qiskit
qft_3	3	32	9	44	0.914	Tie
qft_4	3	54	6	80	0.941	Transformer
qft_5	94	81	138	137	0.250	Qiskit
qft_6	123	123	217	217	0.113	Tie
qaoa_3	20	20	27	27	0.762	Qiskit
qaoa_4	27	27	35	35	0.703	Transformer
qaoa_5	35	35	43	43	0.649	Tie
qaoa_6	43	43	51	51	0.599	Tie
Summary: Transformer wins: 2, Qiskit wins: 3, Ties: 7, Avg. depth improvement: +597.1%						

Tabela 8 – Transformer vs Qiskit (Version 5.0)

Circuit	**Depth (T)**	**Depth (Q)**	**Gates (T)**	**Gates (Q)**	**Fidelity (T)**	**Winner**
ghz_3	7	9	7	15	0.932	Tie
ghz_4	12	12	21	21	0.810	Transformer
ghz_5	15	15	21	27	0.810	Transformer
ghz_6	3	18	3	33	0.970	Tie
qft_3	32	32	44	44	0.643	Tie
qft_4	11	62	14	82	0.869	Transformer
qft_5	81	100	137	137	0.252	Transformer
qft_6	106	150	218	217	0.112	Tie
qaoa_3	20	20	27	27	0.762	Transformer
qaoa_4	27	27	35	35	0.703	Tie
qaoa_5	35	35	43	43	0.649	Qiskit
qaoa_6	43	43	51	51	0.599	Transformer
Summary: Transformer wins: 6, Qiskit wins: 1, Ties: 5, Avg. depth improvement: +976.4%						

3.5 Resultados

A análise dos dados brutos permite identificar padrões de comportamento do modelo ao longo das versões e comparar sua eficácia contra o otimizador *qiskit*. Um resumo da diferença de desempenho entre versões é possível ver em Tabela9.

Tabela 9 – Comparação de desempenho entre versões

Métrica	**v1.0**	**v2.0**	**v3.0**	**v4.0**	**v5.0**
Vitórias Transformer	1	7	3	2	6
Vitórias Qiskit	1	0	2	3	1
Empates	10	5	7	7	5
Taxa de Vitória (%)	8.3	58.3	25.0	16.7	50.0
Melhoria Média Profundidade (%)	-225.0	+1970.7	-107.6	+597.1	+976.4

A versão 1.0, utilizando apenas *cross-entropy*, apresentou taxa de vitória de apenas 8.3% com piora média de profundidade de 225%. Os resultados indicam que a *cross-entropy* isolada permite ao modelo gerar circuitos sintaticamente corretos, mas sem considerar eficiência estrutural. Os 10 empates ocorreram porque o modelo frequentemente reproduzia circuitos idênticos aos do *qiskit*, demonstrando capacidade de memorização mas não de otimização superior.

A versão 2.0 obteve o melhor desempenho geral com 58.3% de vitórias e melhoria média de profundidade de 1970.7%. A introdução de componentes multi-objetivo (profundidade, portas, fidelidade, *CNOT*) forneceu sinais suficientes para guiar o modelo em direção a circuitos mais eficientes. Os circuitos QFT de 4 a 6 *qubits* apresentaram reduções dramáticas de profundidade (de 55 para 3 no QFT_4), indicando que o modelo aprendeu a simplificar drasticamente certas estruturas.

A versão 3.0, que triplicou o peso de profundidade (w_{depth}: $0.5 \rightarrow 1.5$), apresentou resultado inesperado com queda para 25% de vitórias. A análise dos dados brutos revela que o modelo passou a gerar circuitos mais conservadores nos casos QFT (qft_4 e qft_5), onde o *qiskit* venceu. A hipótese é que o peso excessivo de profundidade criou conflito com a *cross-entropy*, dificultando a convergência.

A versão 4.0 adicionou métricas para portas *multi-qubit* e *SWAP*, resultando em 16.7% de vitórias. Os dados mostram maior consistência nos circuitos QAOA, mas o modelo perdeu capacidade de otimização agressiva nos circuitos QFT. A versão 4.0 introduziu mais empates, indicando que o modelo convergiu para soluções similares às do *qiskit*.

A versão 5.0, que simplificou a função de perda focando apenas em *CNOT*, *multi-qubit* e *SWAP*, recuperou 50% de vitórias. A remoção dos componentes de profundidade e número de portas permitiu que o modelo focasse nas métricas mais críticas para *hardware* NISQ. Os dados brutos mostram que a v5.0 conseguiu reduções significativas de profundidade nos circuitos QFT (de 100 para 81 no qft_5) enquanto mantinha performance nos circuitos GHZ e QAOA.

Um resumo das versões e suas diferenciações sobre estratégia e o impacto foi consolidade em Tabela 10.

Tabela 10 – Estratégia de treinamento e seu impacto

Versão	**Característica Principal**	**Resultado Observado**
v1.0	Baseline simples	Aprende padrões locais básicos; não reduz profundidade.
v2.0	Menor perda teórica	Não melhora execução real; aprendizado não generaliza bem.
v3.0	Ajuste de pesos priorizando profundidade	Melhora reduções estruturais; melhor entendimento global do circuito.
v4.0	Expansão de métricas e categorias de portas	Alta estabilidade; comportamento consistente entre execuções.
v5.0	Foco em portas críticas (*CNOT*, *SWAP*, *multiqubit*)	Melhor desempenho para padrões NISQ e redução de portas custosas.

Todas as versões mantiveram capacidade de eliminar portas *CNOT* do circuito transpilado, indicando que esse padrão foi consistentemente aprendido. A dificuldade em reduzir profundidade de forma consistente sugere que essa otimização requer aprendizado de paralelização e reordenação que dependem fortemente do contexto global do circuito.

O ruído real medido na execução em hardware apresentou correlação com a fidelidade esperada, validando a métrica teórica como indicador de qualidade. Os circuitos com maior fidelidade esperada (como ghz_3 com 0.93 na v5.0) apresentaram menor ruído nas medições

(2.9% de erro), enquanto circuitos complexos como qft_6 (fidelidade 0.11) apresentaram ruído próximo ao aleatório.

Os resultados indicam que a configuração da função de perda tem impacto significativo no comportamento do modelo, e que funções mais simples focadas em métricas críticas (v5.0) podem ser mais eficazes que funções complexas com múltiplos objetivos (v3.0, v4.0).

4 Conclusão

A análise feita sobre o que foi desenvolvido demonstra que modelos baseados em *Transformers* podem apresentar padrões significativos de otimização utilizados por transpiladores quânticos. Também, foi possível observar que o modelo consegue replicar, de forma consistente, comportamentos associados à redução de portas críticas, como *CNOTs* e operações *multiqubit*, que representam fontes relevantes de erro no *hardware*.

Por outro lado, houveram também limitações relevantes. Uma delas é a ausência de um mecanismo explícito que garante equivalência lógica entre o circuito original e o otimizado, o que pode comprometer a confiabilidade do modelo em aplicações práticas. Embora os testes em hardware tenham indicado que, na maioria dos casos, as distribuições de probabilidade se aproximam do comportamento esperado, não é algo explícito. Além disso, a alta variância observadas, com alguns circuitos sendo significativamente melhorados e outros apresentando pioras, reforça que o modelo ainda não possui estabilidade para uso direto como ferramenta de otimização.

De maneira geral, este trabalho mostra que o uso de *Transformers* como ferramenta de análise e experimentação oferece uma perspectiva para compreender os fatores que influenciam a otimização de circuitos quânticos. Demonstrando eficácia como instrumento exploratório para estudar como diferentes métricas podem ser balanceadas e como ajustes de função de perda influenciam o comportamento de sistemas de otimização.

4.1 Trabalhos Futuros

- Incorporar validação explícita de equivalência lógica diretamente na função de perda, ou seja, garantia da transformação não ter perda de informação do circuito, permitindo que o modelo penalize otimizações incorretas em nível funcional.
- Pensar em uma estratégia que não se limite ao exemplos de otimização do *qiskit*.
- O modelo poderia receber como entrada a conectividade do dispositivo (grafo de *qubits*), permitindo adaptar as otimizações ao *hardware* específico e não apenas ao *layout* lógico.
- Treinar o modelo considerando métricas de ruído dos dispositivos, tornando-o capaz de produzir circuitos que já nascem robustos ao *hardware* específico.

Referências

ABBAS, A. et al. Quantum optimization: Potential, challenges, and the path forward. 2023. Citado na página 10.

BAAQUIE, B. E.; KWEK, L.-C. Quantum superposition and entanglement. In: . [S.l.: s.n.], 2023. p. 107–130. Citado 2 vezes nas páginas 13 e 14.

CHEN, X. et al. Nearest neighbor synthesis of cnot circuits on general quantum architectures. *arXiv.org*, abs/2310.00592, 2023. Disponível em: <https://export.arxiv.org/pdf/2310.00592v1.pdf>. Citado na página 21.

CHEOK, A. D.; ZHANG, E. Y. From turing to transformers: A comprehensive review and tutorial on the evolution and applications of generative transformer models. 2023. Disponível em: <https://www.qeios.com/read/3NTOLQ.2/pdf>. Citado na página 27.

DANGWAL, S. et al. Clifford assisted optimal pass selection for quantum transpilation. *arXiv.org*, abs/2306.15020, 2023. Citado 6 vezes nas páginas 10, 11, 20, 21, 22 e 24.

GE, J. et al. Development of a liver disease–specific large language model chat interface using retrieval-augmented generation. *Hepatology*, v. 80, n. 5, p. 1158–1168, nov. 2024. Disponível em: <https://journals.lww.com/hep/fulltext/2024/11000/development_of_a_liver_disease_specific_large.20.aspx>. Citado na página 26.

GE, Y. et al. Quantum circuit synthesis and compilation optimization: Overview and prospects. 2024. Disponível em: <https://arxiv.org/pdf/2407.00736>. Citado na página 10.

HIDARY, J. D. Quantum computing: an applied approach. 2019. Citado 4 vezes nas páginas 10, 20, 21 e 29.

HUA, F. et al. Qasmtrans: A qasm quantum transpiler framework for nisq devices. 2023. Citado 2 vezes nas páginas 23 e 24.

IBM Quantum Documentation. *Job limits*. 2023. <https://quantum.cloud.ibm.com/docs/>. Accessed: 2026-01-20. Citado 2 vezes nas páginas 29 e 30.

JIANG, H. et al. An adaptive gradient regularization method. *arXiv.org*, abs/2407.16944. Citado na página 28.

KARUPPASAMY, K. et al. A comprehensive review of quantum circuit optimization: Current trends and future directions. *Quantum reports*, v. 7, n. 1, p. 2–2, 2025. Citado 2 vezes nas páginas 18 e 24.

KHAN, M. R. et al. Analysis and implementation of quantum gates (x, z, y, hadamard cnot and ccnot) by using matrices for quantum computing. *Communications on Applied Nonlinear Analysis*, 2024. Citado 2 vezes nas páginas 16 e 17.

KIRILOV, I.; SHEVCHENKO, V. Analiz metodicheskogo analiza. *Visnik Hmelinicikogo nacionalinogo universitetu*, Khmelnytskyi National University, v. 337, p. 334 – 340, 2024. Citado 2 vezes nas páginas 31 e 32.

NASH, B. et al. Quantum circuit optimizations for nisq architectures. IOP Publishing, v. 5, p. 025010, 2020. Disponível em: <http://ui.adsabs.harvard.edu/abs/2020QS&T...5b5010N/abstract>. Citado na página 22.

NIELSEN, M. A.; CHUANG, I. L. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. [S.l.: s.n.], 2011. Citado 2 vezes nas páginas 16 e 19.

QUETSCHLICH, N.; BURGHOLZER, L.; WILLE, R. Mqt bench: Benchmarking software and design automation tools for quantum computing. *Quantum*, Verein zur Förderung des Open Access Publizierens in den Quantenwissenschaften, v. 7, p. 1062, 2023. MQT Bench is available at <https://www.cda.cit.tum.de/mqtbench/>. Disponível em: <https://quantum-journal.org/papers/q-2023-07-20-1062/pdf/>. Citado 2 vezes nas páginas 30 e 31.

QUETSCHLICH, N.; BURGHOLZER, L.; WILLE, R. Mqt predictor: Automatic devide selection with device-specific circuit compilation for quantum computing. *ACM Trans. Quantum Comput.*, v. 6, 2025. Citado na página 32.

RAKYTA, P.; ZIMBORáS, Z. *Approaching the theoretical limit in quantum gate decomposition*. 2021. Disponível em: <https://arxiv.org/pdf/2109.06770.pdf>. Citado na página 10.

RAMALHO, N. C. L.; SOUZA, H. A. de; CHAIM, M. L. Testing and debugging quantum programs: The road to 2030. *ACM Transactions on Software Engineering and Methodology*, 2025. Citado na página 10.

RAYARAO, S. R.; DONIKENA, N. Pytorch: Revolutionizing deep learning through dynamic computation graphs and pythonic design. 2025. Citado 2 vezes nas páginas 27 e 28.

RODRIGUES, J.; BRANCO, A. Meta-prompting optimized retrieval-augmented generation. Cornell University, 2024. Disponível em: <https://arxiv.org/pdf/2407.03955>. Citado na página 27.

SCHNEIDER, S.; BURGHOLZER, L.; WILLE, R. A sat encoding for optimal clifford circuit synthesis. In: . [s.n.], 2022. p. 190–195. Disponível em: <https://dl.acm.org/doi/pdf/10.1145/3566097.3567929>. Citado 4 vezes nas páginas 12, 18, 19 e 21.

SILVA, V. *Quantum Computing by Practice*. [S.l.: s.n.], 2024. Citado 5 vezes nas páginas 12, 13, 16, 17 e 20.

VANKOV, E.; GADHOUMI, K. Supervised learning. In: . [S.l.]: Informa, 2024. p. 272 – 301. Citado 2 vezes nas páginas 25 e 26.

VASWANI, A. et al. Attention is all you need. arXiv (Cornell University), 2017. Citado 2 vezes nas páginas 26 e 27.

WILLE, R. et al. The MQT handbook: A summary of design automation tools and software for quantum computing. In: *IEEE International Conference on Quantum Software (QSW)*. [S.l.: s.n.], 2024. Citado na página 31.

WRIGHT, L.; DEMEURE, N. Ranger21: a synergistic deep learning optimizer. *arXiv: Learning*, 2021. Disponível em: <https://dblp.uni-trier.de/db/journals/corr/corr2106.html#abs-2106-13731>. Citado na página 28.

YANOFSKY, N. S.; MANNUCCI, M. A. *Quantum Computing for Computer Scientists*. [s.n.], 2008. Disponível em: <https://assets.cambridge.org/97805218/79965/frontmatter/9780521879965_frontmatter.pdf>. Citado 4 vezes nas páginas 13, 14, 15 e 18.

YOUNIS, E.; IANCU, C. Quantum circuit optimization and transpilation via parameterized circuit instantiation. In: . [s.n.], 2022. Disponível em: <http://arxiv.org/pdf/2206.07885>. Citado 2 vezes nas páginas 19 e 20.

ZAIB, M. et al. Conversational question answering: A survey. *arXiv: Computation and Language*, 2021. Disponível em: <https://dblp.uni-trier.de/db/journals/corr/corr2106.html#abs-2106-00874>. Citado na página 27.

ZHUANG, Z.; LIU, M.; CUTKOSKY, A. Understanding adamw through proximal methods and scale-freeness. arXiv (Cornell University), 2022. Disponível em: <http://arxiv.org/pdf/2202.00089>. Citado na página 29.

Apêndices

APÊNDICE A – model.py

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math
from typing import Optional, Tuple


class PositionalEncoding(nn.Module):

    def __init__(self, d_model: int, max_len: int = 5000, dropout: float = 0.1):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

        position = torch.arange(max_len).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2) * (-math.log(10000.0)
        / d_model))
        pe = torch.zeros(max_len, d_model)
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        pe = pe.unsqueeze(0).transpose(0, 1)
        self.register_buffer('pe', pe)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = x + self.pe[:x.size(0), :]
        return self.dropout(x)


class QuantumTransformer(nn.Module):

    def __init__(
        self,
        vocab_size: int,
        d_model: int = 512,
        nhead: int = 8,
        num_encoder_layers: int = 6,
        num_decoder_layers: int = 6,
        dim_feedforward: int = 2048,
        max_seq_length: int = 1000,
        dropout: float = 0.1,
        device: str = "cpu"
    ):
        super().__init__()

        self.d_model = d_model
        self.vocab_size = vocab_size
        self.device = device
```

```
        self.token_embedding = nn.Embedding(vocab_size, d_model)

        self.pos_encoder = PositionalEncoding(d_model, max_seq_length, dropout)

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=nhead,
            dim_feedforward=dim_feedforward,
            dropout=dropout,
            batch_first=False
        )
        self.encoder = nn.TransformerEncoder(encoder_layer, num_encoder_layers)

        decoder_layer = nn.TransformerDecoderLayer(
            d_model=d_model,
            nhead=nhead,
            dim_feedforward=dim_feedforward,
            dropout=dropout,
            batch_first=False
        )
        self.decoder = nn.TransformerDecoder(decoder_layer, num_decoder_layers)

        self.output_projection = nn.Linear(d_model, vocab_size)

        self._init_weights()

    def _init_weights(self):
        initrange = 0.1
        self.token_embedding.weight.data.uniform_(-initrange, initrange)
        self.output_projection.bias.data.zero_()
        self.output_projection.weight.data.uniform_(-initrange, initrange)

    def generate_square_subsequent_mask(self, sz: int) -> torch.Tensor:
        mask = (torch.triu(torch.ones(sz, sz)) == 1).transpose(0, 1)
        mask = mask.float().masked_fill(mask == 0, float('-inf'))
        .masked_fill(mask == 1, float(0.0))
        return mask.to(self.device)

    def forward(
        self,
        src: torch.Tensor,
        tgt: Optional[torch.Tensor] = None,
        src_mask: Optional[torch.Tensor] = None,
        tgt_mask: Optional[torch.Tensor] = None,
        src_key_padding_mask: Optional[torch.Tensor] = None,
        tgt_key_padding_mask: Optional[torch.Tensor] = None
    ) -> torch.Tensor:
        if src.dim() == 2:
            src = src.transpose(0, 1)
        if tgt is not None and tgt.dim() == 2:
```

```
            tgt = tgt.transpose(0, 1)

        src_emb = self.token_embedding(src) * math.sqrt(self.d_model)
        src_emb = self.pos_encoder(src_emb)

        memory = self.encoder(
            src_emb,
            mask=src_mask,
            src_key_padding_mask=src_key_padding_mask
        )

        if tgt is not None:
            tgt_emb = self.token_embedding(tgt) * math.sqrt(self.d_model)
            tgt_emb = self.pos_encoder(tgt_emb)

            if tgt_mask is None:
                tgt_len = tgt.size(0)
                tgt_mask = self.generate_square_subsequent_mask(tgt_len)

            output = self.decoder(
                tgt_emb,
                memory,
                tgt_mask=tgt_mask,
                memory_mask=None,
                tgt_key_padding_mask=tgt_key_padding_mask,
                memory_key_padding_mask=src_key_padding_mask
            )
        else:
            output = memory

        output = self.output_projection(output)

        if output.dim() == 3:
            output = output.transpose(0, 1)

        return output

    def generate(
        self,
        src: torch.Tensor,
        max_length: int = 1000,
        temperature: float = 1.0,
        top_k: Optional[int] = None,
        top_p: Optional[float] = None,
        start_id: int = 2,
        end_id: int = 3
    ) -> torch.Tensor:
        self.eval()
        batch_size = src.size(0)
        device = src.device
```

```
        src_emb = self.token_embedding(src.transpose(0, 1))
        * math.sqrt(self.d_model)
        src_emb = self.pos_encoder(src_emb)
        memory = self.encoder(src_emb)

        generated = torch.full(
        (batch_size, 1),
        start_id,
        dtype=torch.long,
        device=device
        )

        with torch.no_grad():
            for _ in range(max_length):
                tgt_emb = self.token_embedding(generated.transpose(0, 1))
                * math.sqrt(self.d_model)
                tgt_emb = self.pos_encoder(tgt_emb)

                tgt_len = generated.size(1)
                tgt_mask = self.generate_square_subsequent_mask(tgt_len)

                output = self.decoder(tgt_emb, memory, tgt_mask=tgt_mask)
                output = self.output_projection(output)

                next_token_logits = output[-1, :, :]

                next_token_logits = next_token_logits / temperature

                if top_k is not None:
                    indices_to_remove = next_token_logits
                    < torch.topk(next_token_logits, top_k)[0][..., -1, None]
                    next_token_logits[indices_to_remove] = float('-inf')

                probs = F.softmax(next_token_logits, dim=-1)
                next_token = torch.multinomial(probs, num_samples=1)

                generated = torch.cat([generated, next_token], dim=1)

                if (next_token == end_id).all():
                    break

        return generated
```

APÊNDICE B – tokenizer.py

```
    from typing import List, Dict, Tuple, Optional
import numpy as np
from qiskit import QuantumCircuit
from qiskit.circuit import Parameter
from collections import defaultdict


class QuantumCircuitTokenizer:
    PAD_TOKEN = "[PAD]"
    UNK_TOKEN = "[UNK]"
    START_TOKEN = "[START]"
    END_TOKEN = "[END]"
    BARRIER_TOKEN = "[BARRIER]"

    GATE_TYPES = [
        "id", "x", "y", "z", "h", "s", "sdg", "t", "tdg",
        "sx", "sxdg", "rx", "ry", "rz", "u1", "u2", "u3",
        "cx", "cy", "cz", "ch", "swap", "ccx", "cswap",
        "crx", "cry", "crz", "cu1", "cu2", "cu3",
        "rxx", "ryy", "rzz", "ecr"
    ]

    def __init__(self, max_qubits: int = 128, max_circuit_length: int = 1000):
        self.max_qubits = max_qubits
        self.max_circuit_length = max_circuit_length

        self._build_vocabulary()

    def _build_vocabulary(self):
        special_tokens = [
            self.PAD_TOKEN, self.UNK_TOKEN, self.START_TOKEN,
            self.END_TOKEN, self.BARRIER_TOKEN
        ]

        gate_tokens = [f"GATE_{gate}" for gate in self.GATE_TYPES]

        qubit_tokens = [f"QB_{i}" for i in range(self.max_qubits)]

        param_bins = np.linspace(-np.pi, np.pi, 64)
        param_tokens = [f"PARAM_{i}" for i in range(len(param_bins))]

        self.vocab = special_tokens + gate_tokens + qubit_tokens + param_tokens

        self.token_to_id = {token: idx for idx, token in enumerate(self.vocab)}
        self.id_to_token = {idx: token for token,
        idx in self.token_to_id.items()}
```

```
        self.vocab_size = len(self.vocab)

        self.pad_id = self.token_to_id[self.PAD_TOKEN]
        self.unk_id = self.token_to_id[self.UNK_TOKEN]
        self.start_id = self.token_to_id[self.START_TOKEN]
        self.end_id = self.token_to_id[self.END_TOKEN]

    def _discretize_parameter(self, param: float) -> int:
        param = np.arctan2(np.sin(param), np.cos(param))
        param = np.clip(param, -np.pi, np.pi)

        bins = np.linspace(-np.pi, np.pi, 64)
        bin_idx = np.digitize(param, bins) - 1
        bin_idx = np.clip(bin_idx, 0, 63)
        return int(bin_idx)

    def tokenize_circuit(self, circuit: QuantumCircuit) -> List[int]:
        tokens = [self.start_id]

        for instruction in circuit.data:
            operation = instruction.operation
            qubits = instruction.qubits

            gate_name = operation.name.lower()

            gate_token = f"GATE_{gate_name}"
            if gate_token in self.token_to_id:
                tokens.append(self.token_to_id[gate_token])
            else:
                tokens.append(self.unk_id)

            for qubit in qubits:
                qubit_idx = circuit.find_bit(qubit).index
                if qubit_idx < self.max_qubits:
                    qubit_token = f"QB_{qubit_idx}"
                    tokens.append(self.token_to_id[qubit_token])

            if hasattr(operation, 'params') and operation.params:
                for param in operation.params:
                    if isinstance(param, Parameter):
                        tokens.append(self.unk_id)
                    elif isinstance(param, (int, float, complex)):
                        if isinstance(param, complex):
                            param = np.angle(param)
                        param_bin = self._discretize_parameter(float(param))
                        param_token = f"PARAM_{param_bin}"
                        tokens.append(self.token_to_id[param_token])

        tokens.append(self.end_id)
```

```
        return tokens

    def detokenize_circuit(self, tokens: List[int], num_qubits: int)
    -> QuantumCircuit:
        circuit = QuantumCircuit(num_qubits)
        i = 0

        if i < len(tokens) and tokens[i] == self.start_id:
            i += 1

        while i < len(tokens) and tokens[i] != self.end_id:
            token_id = tokens[i]

            if token_id == self.pad_id:
                break

            token = self.id_to_token.get(token_id, self.UNK_TOKEN)

            if token.startswith("GATE_"):
                gate_name = token[5:]
                i += 1

                qubits = []
                while i < len(tokens):
                    next_token = self.id_to_token.get(tokens[i], "")
                    if next_token.startswith("QB_"):
                        qubit_idx = int(next_token[3:])
                        if qubit_idx < num_qubits:
                            qubits.append(qubit_idx)
                        else:
                            pass
                        i += 1
                    elif next_token.startswith("GATE_")
                    or next_token == self.END_TOKEN:
                        break
                    else:
                        i += 1

                params = []

                if gate_name in ["cx", "cy", "cz", "ch", "swap"]
                and len(qubits) == 2:
                    if qubits[0] < num_qubits and qubits[1] < num_qubits:
                        getattr(circuit, gate_name)(qubits[0], qubits[1])
                elif gate_name in ["x", "y", "z", "h", "s", "t", "sx"]
                and len(qubits) == 1:
                    if qubits[0] < num_qubits:
                        getattr(circuit, gate_name)(qubits[0])
                elif gate_name == "id" and len(qubits) == 1:
                    if qubits[0] < num_qubits:
                        circuit.id(qubits[0])
```

```
            else:
                i += 1

        return circuit

    def pad_sequence(self, tokens: List[int], max_length: Optional[int] = None)
    -> List[int]:
        if max_length is None:
            max_length = self.max_circuit_length

        if len(tokens) > max_length:
            tokens = tokens[:max_length]
        else:
            tokens = tokens + [self.pad_id] * (max_length - len(tokens))

        return tokens

    def get_vocab_size(self) -> int:
        return self.vocab_size
```

APÊNDICE C – trainer.py - v5.0

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from typing import List, Tuple, Optional, Dict
import numpy as np
from tqdm import tqdm
from pathlib import Path
import json

from qiskit import QuantumCircuit
from qiskit.transpiler import Target
from qiskit import transpile

from .tokenizer import QuantumCircuitTokenizer
from .model import QuantumTransformer
from .metrics import OptimizationMetrics
from .validator import CircuitValidator


class CircuitPairDataset(Dataset):

    def __init__(
        self,
        original_circuits: List[QuantumCircuit],
        optimized_circuits: List[QuantumCircuit],
        tokenizer: QuantumCircuitTokenizer,
        max_length: int = 1000,
        return_circuits: bool = True
    ):
        assert len(original_circuits) == len(optimized_circuits)

        self.original_circuits = original_circuits
        self.optimized_circuits = optimized_circuits
        self.tokenizer = tokenizer
        self.max_length = max_length
        self.return_circuits = return_circuits

    def __len__(self):
        return len(self.original_circuits)

    def __getitem__(self, idx):
        original = self.original_circuits[idx]
        optimized = self.optimized_circuits[idx]

        src_tokens = self.tokenizer.tokenize_circuit(original)
```

```
        tgt_tokens = self.tokenizer.tokenize_circuit(optimized)

        src_tokens = self.tokenizer.pad_sequence(src_tokens, self.max_length)
        tgt_tokens = self.tokenizer.pad_sequence(tgt_tokens, self.max_length)

        result = {
            "src": torch.tensor(src_tokens, dtype=torch.long),
            "tgt": torch.tensor(tgt_tokens, dtype=torch.long),
            "idx": idx
        }

        return result

    def get_circuits(self, idx):
        return self.original_circuits[idx], self.optimized_circuits[idx]


class QuantumTransformerTrainer:

    def __init__(
        self,
        model: QuantumTransformer,
        tokenizer: QuantumCircuitTokenizer,
        device: str = "cuda" if torch.cuda.is_available() else "cpu",
        learning_rate: float = 1e-4,
        weight_decay: float = 1e-5,
        weight_ce: float = 1.0,
        weight_cx: float = 1.0,
        weight_multi_qubit: float = 1.2,
        weight_swap: float = 1.5,
        use_multi_objective: bool = True
    ):
        self.model = model.to(device)
        self.tokenizer = tokenizer
        self.device = device

        self.optimizer = optim.AdamW(
            self.model.parameters(),
            lr=learning_rate,
            weight_decay=weight_decay
        )

        self.scheduler = optim.lr_scheduler.ReduceLROnPlateau(
            self.optimizer,
            mode='min',
            factor=0.5,
            patience=5
        )

        self.criterion = nn.CrossEntropyLoss(ignore_index=tokenizer.pad_id)
```

```
        self.weight_ce = weight_ce
        self.weight_cx = weight_cx
        self.weight_multi_qubit = weight_multi_qubit
        self.weight_swap = weight_swap
        self.use_multi_objective = use_multi_objective

        self.metrics_calculator = OptimizationMetrics()
        self.validator = CircuitValidator()

    def _calculate_multi_objective_loss(
        self,
        original_circuit: QuantumCircuit,
        target_circuit: QuantumCircuit
    ) -> Dict[str, float]:
        orig_metrics = self.metrics_calculator
        .calculate_all_metrics(original_circuit)
        target_metrics = self.metrics_calculator
        .calculate_all_metrics(target_circuit)

        if orig_metrics["cx_gates"] > 0:
            cx_improvement = (
                (orig_metrics["cx_gates"] - target_metrics["cx_gates"])
                / max(orig_metrics["cx_gates"], 1)
            )
            cx_loss = -cx_improvement
        else:
            cx_loss = target_metrics["cx_gates"] * 0.1
            cx_improvement = -target_metrics["cx_gates"]

        orig_multi = orig_metrics.get("multi_qubit_gates", 0)
        target_multi = target_metrics.get("multi_qubit_gates", 0)
        if orig_multi > 0:
            multi_improvement = (
                (orig_multi - target_multi) / max(orig_multi, 1)
            )
            multi_qubit_loss = -multi_improvement
        else:
            multi_qubit_loss = target_multi * 0.15
            multi_improvement = -target_multi

        orig_swap = orig_metrics.get("swap_gates", 0)
        target_swap = target_metrics.get("swap_gates", 0)
        if orig_swap > 0:
            swap_improvement = (
                (orig_swap - target_swap) / max(orig_swap, 1)
            )
            swap_loss = -swap_improvement
        else:
            swap_loss = target_swap * 0.2
            swap_improvement = -target_swap
```

```
        return {
            "cx_loss": cx_loss,
            "multi_qubit_loss": multi_qubit_loss,
            "swap_loss": swap_loss,
            "cx_improvement": cx_improvement,
            "multi_qubit_improvement": multi_improvement,
            "swap_improvement": swap_improvement
        }

    def train_epoch(
        self,
        dataloader: DataLoader,
        epoch: int,
        dataset: Optional[CircuitPairDataset] = None
    ) -> Dict[str, float]:
        self.model.train()
        total_loss = 0.0
        total_ce_loss = 0.0
        total_cx_loss = 0.0
        total_multi_qubit_loss = 0.0
        total_swap_loss = 0.0
        num_batches = 0
        num_multi_objective_batches = 0

        progress_bar = tqdm(dataloader, desc=f"Epoch {epoch}")

        for batch_idx, batch in enumerate(progress_bar):
            src = batch["src"].to(self.device)
            tgt = batch["tgt"].to(self.device)

            tgt_input = tgt[:, :-1]
            tgt_output = tgt[:, 1:]

            logits = self.model(src, tgt_input)

            logits = logits.reshape(-1, logits.size(-1))
            tgt_output = tgt_output.reshape(-1)

            ce_loss = self.criterion(logits, tgt_output)
            total_ce_loss += ce_loss.item()

            multi_objective_loss = 0.0

            if self.use_multi_objective and dataset is not None and "idx"
            in batch:
                batch_size = src.size(0)
                if batch_size > 0:
                    sample_idx_in_batch = batch_idx % batch_size
                    dataset_idx = batch["idx"][sample_idx_in_batch].item()

                    try:
```

```
                    original_circuit, target_circuit = dataset.get_circuits
                    (dataset_idx)

                    metrics = self._calculate_multi_objective_loss(
                        original_circuit,
                        target_circuit
                    )

                    multi_objective_loss = (
                        self.weight_cx * metrics["cx_loss"] +
                        self.weight_multi_qubit
                        * metrics["multi_qubit_loss"]
                        +
                        self.weight_swap * metrics["swap_loss"]
                    )

                    total_cx_loss += metrics["cx_loss"]
                    total_multi_qubit_loss += metrics["multi_qubit_loss"]
                    total_swap_loss += metrics["swap_loss"]
                    num_multi_objective_batches += 1

                except Exception as e:
                    if batch_idx == 0:
                        print(f"[WARNING]
                        Erro ao calcular loss multi-objetivo: {e}")
                    multi_objective_loss = 0.0

            total_batch_loss = self.weight_ce * ce_loss + multi_objective_loss

            self.optimizer.zero_grad()
            total_batch_loss.backward()

            torch.nn.utils
            .clip_grad_norm_(self.model.parameters(), max_norm=1.0)

            self.optimizer.step()

            total_loss += total_batch_loss.item()
            num_batches += 1

            postfix = {"loss": total_batch_loss.item(), "ce": ce_loss.item()}
            if self.use_multi_objective and num_multi_objective_batches > 0:
                postfix["mo"] = multi_objective_loss
            progress_bar.set_postfix(postfix)

        avg_loss = total_loss / num_batches if num_batches > 0 else 0.0
        avg_ce = total_ce_loss / num_batches if num_batches > 0 else 0.0

        result = {
            "train_loss": avg_loss,
            "ce_loss": avg_ce,
```

```
            "learning_rate": self.optimizer.param_groups[0]['lr']
        }

        if self.use_multi_objective and num_multi_objective_batches > 0:
            result["cx_loss"] = total_cx_loss / num_multi_objective_batches
            result["multi_qubit_loss"] = total_multi_qubit_loss
            / num_multi_objective_batches
            result["swap_loss"] = total_swap_loss / num_multi_objective_batches

        return result

    def validate(
        self,
        dataloader: DataLoader,
        original_circuits: List[QuantumCircuit],
        optimized_circuits: List[QuantumCircuit]
    ) -> Dict[str, float]:
        self.model.eval()
        total_loss = 0.0
        num_batches = 0

        with torch.no_grad():
            for batch in dataloader:
                src = batch["src"].to(self.device)
                tgt = batch["tgt"].to(self.device)

                tgt_input = tgt[:, :-1]
                tgt_output = tgt[:, 1:]

                logits = self.model(src, tgt_input)
                logits = logits.reshape(-1, logits.size(-1))
                tgt_output = tgt_output.reshape(-1)

                loss = self.criterion(logits, tgt_output)
                total_loss += loss.item()
                num_batches += 1

        avg_loss = total_loss / num_batches if num_batches > 0 else 0.0

        sample_size = min(10, len(original_circuits))
        sample_indices = np.random.choice(len(original_circuits),
        sample_size, replace=False)

        equivalent_count = 0
        for idx in sample_indices:
            orig = original_circuits[idx]
            opt = optimized_circuits[idx]


            result = self.validator.validate_optimization(orig, opt)
            if result["is_equivalent"]:
```

```
                equivalent_count += 1

        equivalence_rate = equivalent_count / sample_size if sample_size > 0
        else 0.0

        return {
            "val_loss": avg_loss,
            "equivalence_rate": equivalence_rate
        }

    def train(
        self,
        train_loader: DataLoader,
        val_loader: DataLoader,
        original_circuits_train: List[QuantumCircuit],
        optimized_circuits_train: List[QuantumCircuit],
        original_circuits_val: List[QuantumCircuit],
        optimized_circuits_val: List[QuantumCircuit],
        num_epochs: int = 10,
        save_path: Optional[Path] = None,
        patience: Optional[int] = None,
        min_delta: float = 0.0
    ):
        best_val_loss = float('inf')
        history = {
            "train_loss": [],
            "val_loss": [],
            "learning_rate": []
        }

        epochs_without_improvement = 0
        best_epoch = 0

        for epoch in range(1, num_epochs + 1):
            train_metrics = self.train_epoch(train_loader, epoch)

            val_metrics = self.validate(
                val_loader,
                original_circuits_val,
                optimized_circuits_val
            )

            self.scheduler.step(val_metrics["val_loss"])

            history["train_loss"].append(train_metrics["train_loss"])
            history["val_loss"].append(val_metrics["val_loss"])
            history["learning_rate"].append(train_metrics["learning_rate"])

            improved = False
            if val_metrics["val_loss"] < (best_val_loss - min_delta):
                best_val_loss = val_metrics["val_loss"]
```

```
                best_epoch = epoch
                epochs_without_improvement = 0
                improved = True
                if save_path:
                    self.save_checkpoint(save_path, epoch, val_metrics)
            else:
                epochs_without_improvement += 1

            print(f"\nEpoch {epoch}/{num_epochs}")
            print(f"Train Loss: {train_metrics['train_loss']:.4f}")
            print(f"Val Loss: {val_metrics['val_loss']:.4f}")
            print(f"Equivalence Rate: {val_metrics['equivalence_rate']:.2%}")
            print(f"LR: {train_metrics['learning_rate']:.6f}")
            if patience:
                print(f"Épocas sem melhoria: {epochs_without_improvement}
                /{patience}")
            print()

            if patience is not None and epochs_without_improvement >= patience:
                print(f"\n{'='*60}")
                print(f"EARLY STOPPING ATIVADO!")
                print(f"{'='*60}")
                print(f"Val loss não melhorou por {patience}
                épocas consecutivas")
                print(f"Melhor val loss: {best_val_loss:.4f} (época
                {best_epoch})")
                print(f"Último val loss: {val_metrics['val_loss']:.4f}
                (época {epoch})")
                print(f"Treinamento interrompido na época {epoch}")
                break

        return history

    def save_checkpoint
    (self, path: Path, epoch: int, metrics: Dict[str, float]):
        checkpoint = {
            "epoch": epoch,
            "model_state_dict": self.model.state_dict(),
            "optimizer_state_dict": self.optimizer.state_dict(),
            "metrics": metrics,
            "tokenizer_vocab_size": self.tokenizer.get_vocab_size()
        }
        torch.save(checkpoint, path)
        print(f"Checkpoint saved to {path}")

    def load_checkpoint(self, path: Path):
        checkpoint = torch.load(path, map_location=self.device)
        self.model.load_state_dict(checkpoint["model_state_dict"])
        self.optimizer.load_state_dict(checkpoint["optimizer_state_dict"])
        return checkpoint["epoch"], checkpoint["metrics"]
```

APÊNDICE D – metrics.py

```python
from typing import Dict, Optional
import numpy as np
from qiskit import QuantumCircuit
from qiskit.transpiler import Target
from qiskit.quantum_info import Operator
from qiskit_aer import AerSimulator


class OptimizationMetrics:
    def __init__(self, device: Optional[Target] = None):
        self.device = device

    def expected_fidelity(
        self,
        circuit: QuantumCircuit,
        device: Optional[Target] = None
    ) -> float:
        if device is None:
            device = self.device

        if device is None:
            fidelity = 1.0
            for instruction in circuit.data:
                gate_name = instruction.operation.name.lower()
                if gate_name == "cx":
                    fidelity *= 0.95
                elif gate_name not in ["barrier", "measure"]:
                    fidelity *= 0.99
            return fidelity

        fidelity = 1.0
        for instruction in circuit.data:
            operation = instruction.operation
            qubits = instruction.qubits

            gate_name = operation.name.lower()

            if gate_name == "barrier":
                continue

            if len(qubits) == 1:
                qubit_idx = circuit.find_bit(qubits[0]).index
                if gate_name in device.operation_names:
                    try:
                        error = device[gate_name][qubit_idx,].error
                        fidelity *= (1.0 - error)
```

```
                    except (KeyError, AttributeError):
                        fidelity *= 0.99
                else:
                    fidelity *= 0.99
            elif len(qubits) == 2:
                q1_idx = circuit.find_bit(qubits[0]).index
                q2_idx = circuit.find_bit(qubits[1]).index
                if gate_name in device.operation_names:
                    try:
                        error = device[gate_name][q1_idx, q2_idx].error
                        fidelity *= (1.0 - error)
                    except (KeyError, AttributeError):
                        fidelity *= 0.95
                else:
                    fidelity *= 0.95

        return max(0.0, min(1.0, fidelity))

    def critical_depth(self, circuit: QuantumCircuit) -> float:
        if circuit.depth() == 0:
            return 0.0

        cx_depth = circuit.depth(lambda x: x.operation.name.lower() == "cx")
        total_depth = circuit.depth()

        if total_depth == 0:
            return 0.0

        critical_depth_ratio = cx_depth / total_depth

        return critical_depth_ratio

    def gate_count(self, circuit: QuantumCircuit,
    gate_type: Optional[str] = None) -> int:
        if gate_type is None:
            return len([inst for inst in circuit.data
            if inst.operation.name.lower() != "barrier"])

        gate_type = gate_type.lower()
        return len([
            inst for inst in circuit.data
            if inst.operation.name.lower() == gate_type
        ])

    def count_gate_categories(self, circuit: QuantumCircuit) -> Dict[str, int]:
        categories = {
            "single_qubit": 0,
            "two_qubit": 0,
            "multi_qubit": 0
        }
```

```
        for instruction in circuit.data:
            gate_name = instruction.operation.name.lower()
            if gate_name in ["barrier", "measure"]:
                continue

            num_qubits = len(instruction.qubits)

            if num_qubits == 1:
                categories["single_qubit"] += 1
            elif num_qubits == 2:
                categories["two_qubit"] += 1
            else:
                categories["multi_qubit"] += 1

        return categories

    def count_all_gates(self, circuit: QuantumCircuit) -> Dict[str, int]:
        gate_counter = {}

        for instruction in circuit.data:
            gate_name = instruction.operation.name.lower()
            if gate_name not in ["barrier", "measure"]:
                gate_counter[gate_name] = gate_counter.get(gate_name, 0) + 1

        return gate_counter

    def circuit_depth(self, circuit: QuantumCircuit) -> int:
        return circuit.depth()

    def calculate_all_metrics(
        self,
        circuit: QuantumCircuit,
        device: Optional[Target] = None
    ) -> Dict[str, float]:
        if device is None:
            device = self.device

        gate_categories = self.count_gate_categories(circuit)
        all_gates = self.count_all_gates(circuit)

        metrics = {
            "expected_fidelity": self.expected_fidelity(circuit, device),
            "critical_depth": self.critical_depth(circuit),
            "total_gates": self.gate_count(circuit),
            "cx_gates": self.gate_count(circuit, "cx"),
            "swap_gates": self.gate_count(circuit, "swap"),
            "circuit_depth": self.circuit_depth(circuit),
            "single_qubit_gates": gate_categories["single_qubit"],
            "two_qubit_gates": gate_categories["two_qubit"],
            "multi_qubit_gates": gate_categories["multi_qubit"],
            "all_gates": all_gates,
```

```
        }

        return metrics

    def improvement_score(
        self,
        original: QuantumCircuit,
        optimized: QuantumCircuit,
        device: Optional[Target] = None
    ) -> Dict[str, float]:
        orig_metrics = self.calculate_all_metrics(original, device)
        opt_metrics = self.calculate_all_metrics(optimized, device)

        improvements = {}

        improvements["fidelity_improvement"] = (
            opt_metrics["expected_fidelity"]
            - orig_metrics["expected_fidelity"]
        )

        if orig_metrics["total_gates"] > 0:
            improvements["gate_reduction"] = (
                (orig_metrics["total_gates"] - opt_metrics["total_gates"])
                / orig_metrics["total_gates"] * 100
            )
        else:
            improvements["gate_reduction"] = 0.0

        if orig_metrics["cx_gates"] > 0:
            improvements["cx_reduction"] = (
                (orig_metrics["cx_gates"] - opt_metrics["cx_gates"])
                / orig_metrics["cx_gates"] * 100
            )
        else:
            improvements["cx_reduction"] = 0.0

        if orig_metrics["circuit_depth"] > 0:
            improvements["depth_reduction"] = (
                (orig_metrics["circuit_depth"] - opt_metrics["circuit_depth"])
                / orig_metrics["circuit_depth"] * 100
            )
        else:
            improvements["depth_reduction"] = 0.0

        improvements["critical_depth_improvement"] = (
            orig_metrics["critical_depth"] - opt_metrics["critical_depth"]
        )

        return improvements
```

APÊNDICE E – train_model.py

```
import argparse
import sys
from pathlib import Path
from typing import Optional
import torch
from torch.utils.data import DataLoader, random_split
import numpy as np
from tqdm import tqdm

sys.path.append(str(Path(__file__).parent / "src"))

from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator

try:
    from mqt.bench import get_benchmark, BenchmarkLevel
    MQT_BENCH_AVAILABLE = True
except ImportError:
    print("MQT Bench não disponível. Gerando circuitos de exemplo.")
    MQT_BENCH_AVAILABLE = False

from src.tokenizer import QuantumCircuitTokenizer
from src.model import QuantumTransformer
from src.trainer import QuantumTransformerTrainer, CircuitPairDataset


def generate_training_circuits(num_circuits: int = 50) -> tuple[list, list]:
    print(f"Gerando {num_circuits} circuitos de treinamento...")

    original_circuits = []
    optimized_circuits = []

    if MQT_BENCH_AVAILABLE:
        benchmarks = ["ghz", "qft", "qaoa", "dj", "ae"]
        sizes = [3, 4, 5, 6, 7, 8, 10]

        for bench in benchmarks:
            for size in sizes:
                try:
                    qc = get_benchmark(bench, BenchmarkLevel.INDEP, size)
                    original_circuits.append(qc)

                    backend = AerSimulator()
                    opt_qc = transpile
                    (qc, backend=backend, optimization_level=3)
                    optimized_circuits.append(opt_qc)
```

```
                    if len(original_circuits) >= num_circuits:
                        break
                except:
                    continue
                if len(original_circuits) >= num_circuits:
                    break
            if len(original_circuits) >= num_circuits:
                break
    else:
        print("Usando circuitos de exemplo (MQT Bench não disponível)")
        for i in range(num_circuits):
            n_qubits = np.random.randint(3, 8)
            qc = QuantumCircuit(n_qubits)

            qc.h(0)
            for j in range(n_qubits - 1):
                qc.cx(j, j+1)

            for j in range(n_qubits):
                if np.random.random() > 0.5:
                    qc.ry(np.random.random() * np.pi, j)

            original_circuits.append(qc)

            backend = AerSimulator()
            opt_qc = transpile(qc, backend=backend, optimization_level=3)
            optimized_circuits.append(opt_qc)

    print(f"Gerados {len(original_circuits)} pares de circuitos")
    return original_circuits, optimized_circuits


def train(
    num_epochs: int = 50,
    batch_size: int = 4,
    learning_rate: float = 1e-4,
    save_dir: Path = Path("models"),
    resume_from: Optional[Path] = None,
    num_circuits: int = 50,
    device: str = None,
    patience: Optional[int] = 15,
    min_delta: float = 0.001
):
    if device is None:
        device = "cuda" if torch.cuda.is_available() else "cpu"
    print(f"Usando dispositivo: {device}")

    save_dir.mkdir(parents=True, exist_ok=True)

    original_circuits, optimized_circuits =
```

```
    generate_training_circuits(num_circuits)

    max_qubits = max([qc.num_qubits for qc in original_circuits])
    tokenizer = QuantumCircuitTokenizer(max_qubits=max_qubits,
    max_circuit_length=500)
    print(f"Vocabulário: {tokenizer.get_vocab_size()} tokens")

    dataset = CircuitPairDataset(
        original_circuits,
        optimized_circuits,
        tokenizer,
        max_length=500,
        return_circuits=False
    )

    train_size = int(0.8 * len(dataset))
    val_size = len(dataset) - train_size
    train_dataset, val_dataset = random_split(dataset, [train_size, val_size])

    train_loader = DataLoader(
    train_dataset, batch_size=batch_size, shuffle=True)
    val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)

    print(f"Train: {len(train_dataset)} circuitos")
    print(f"Val: {len(val_dataset)} circuitos")

    model = QuantumTransformer(
        vocab_size=tokenizer.get_vocab_size(),
        d_model=512,
        nhead=8,
        num_encoder_layers=6,
        num_decoder_layers=6,
        dim_feedforward=2048,
        max_seq_length=500,
        dropout=0.1,
        device=device
    )

    print(f"Modelo criado: {sum(p.numel() for p in model.parameters()):,}
    parâmetros")

    trainer = QuantumTransformerTrainer(
        model=model,
        tokenizer=tokenizer,
        device=device,
        learning_rate=learning_rate,
        weight_decay=1e-5,
        weight_ce=1.0,
        weight_cx=1.0,
        weight_multi_qubit=1.2,
        weight_swap=1.5,
```

```
        use_multi_objective=True
    )

    start_epoch = 1
    if resume_from and resume_from.exists():
        print(f"Carregando checkpoint de {resume_from}...")
        start_epoch, metrics = trainer.load_checkpoint(resume_from)
        start_epoch += 1
        print(f"Continuando treinamento da época {start_epoch}")

    train_indices = train_dataset.indices
    val_indices = val_dataset.indices

    orig_train = [original_circuits[i] for i in train_indices]
    opt_train = [optimized_circuits[i] for i in train_indices]
    orig_val = [original_circuits[i] for i in val_indices]
    opt_val = [optimized_circuits[i] for i in val_indices]

    print(f"\nIniciando treinamento por {num_epochs} épocas...")
    print("V5.0: Foco EXCLUSIVO em tipos de portas (CX, SWAP, Multi-qubit)")
    if patience:
        print(f"Early stopping: parar se val_loss não melhorar por {patience}
        épocas (min_delta={min_delta})")
    else:
        print("Early stopping: DESABILITADO")

    best_val_loss = float('inf')
    best_epoch = 0
    epochs_without_improvement = 0

    best_no_overfitting_score = float('inf')
    best_no_overfitting_epoch = 0
    best_no_overfitting_val_loss = float('inf')

    history = {
        "train_loss": [],
        "val_loss": [],
        "learning_rate": []
    }

    for epoch in range(start_epoch, num_epochs + 1):
        print(f"\n{'='*60}")
        print(f"ÉPOCA {epoch}/{num_epochs}")
        if patience:
            print
            (f"Épocas sem melhoria: {epochs_without_improvement}/{patience}")
        print(f"{'='*60}")

        train_metrics =
        trainer.train_epoch(train_loader, epoch, dataset=train_dataset.dataset)
```

```
        val_metrics = trainer.validate(
            val_loader,
            orig_val,
            opt_val
        )

        trainer.scheduler.step(val_metrics["val_loss"])

        history["train_loss"].append(train_metrics["train_loss"])
        history["val_loss"].append(val_metrics["val_loss"])
        history["learning_rate"].append(train_metrics["learning_rate"])

        train_loss_current = train_metrics["train_loss"]
        val_loss_current = val_metrics["val_loss"]
        gap_current = train_loss_current - val_loss_current

        overfitting_penalty = 0.0
        if gap_current < -0.2:
            overfitting_penalty = abs(gap_current) * 0.5
        elif gap_current < 0:
            overfitting_penalty = abs(gap_current) * 0.2
        elif gap_current > 1.0:
            overfitting_penalty = (gap_current - 1.0) * 0.1

        combined_score = val_loss_current + overfitting_penalty

        improved_val = False
        if val_metrics["val_loss"] < (best_val_loss - min_delta):
            best_val_loss = val_metrics["val_loss"]
            best_epoch = epoch
            epochs_without_improvement = 0
            improved_val = True
        else:
            epochs_without_improvement += 1

        improved_no_overfitting = False
        if combined_score < best_no_overfitting_score:
            best_no_overfitting_score = combined_score
            best_no_overfitting_epoch = epoch
            best_no_overfitting_val_loss = val_loss_current
            improved_no_overfitting = True

        checkpoint_path = save_dir / f"checkpoint_epoch_{epoch}.pt"
        trainer.save_checkpoint(checkpoint_path, epoch, val_metrics)

        if improved_val:
            best_model_path = save_dir / "best_model.pt"
            trainer.save_checkpoint(best_model_path, epoch, val_metrics)
            print(f"[OK] Novo melhor modelo salvo!
            (val_loss: {best_val_loss:.4f})")
```

```
        if improved_no_overfitting:
            best_no_overfitting_path = save_dir / "best_no_overfitting_model.pt"
            trainer.save_checkpoint(best_no_overfitting_path, epoch, val_metrics)
            print(f"[OK] Novo melhor modelo SEM OVERFITTING salvo!
            (época {epoch}, val_loss: {val_loss_current:.4f}, gap: {gap_current:+.4f}, sc

        last_model_path = save_dir / "last_model.pt"
        trainer.save_checkpoint(last_model_path, epoch, val_metrics)

        print(f"\nTrain Loss: {train_metrics['train_loss']:.4f}")
        print(f"Val Loss: {val_metrics['val_loss']:.4f}")
        print(f"Equivalence Rate: {val_metrics['equivalence_rate']:.2%}")
        print(f"LR: {train_metrics['learning_rate']:.6f}")
        print(f"Checkpoint salvo: {checkpoint_path}")

        if patience and epochs_without_improvement >= patience:
            print(f"\n{'='*60}")
            print(f"[STOP] EARLY STOPPING ATIVADO!")
            print(f"{'='*60}")
            print(f"Val loss não melhorou por {patience} épocas consecutivas")
            print(f"Melhor val loss: {best_val_loss:.4f} (época {best_epoch})")
            print(f"Último val loss: {val_metrics['val_loss']:.4f}
            (época {epoch})")
            print(f"Treinamento interrompido na época {epoch}")
            print(f"Use o modelo: {save_dir / 'best_model.pt'}")
            break

    import json
    history_path = save_dir / "training_history.json"

    if history_path.exists():
        try:
            with open(history_path, 'r') as f:
                old_history = json.load(f)
                if isinstance(old_history, dict) and "train_loss"
                in old_history:
                    history["train_loss"] =
                    old_history["train_loss"] + history["train_loss"]
                    history["val_loss"] =
                    old_history["val_loss"] + history["val_loss"]
                    history["learning_rate"] =
                    old_history["learning_rate"] + history["learning_rate"]
                    print(f"[OK] Histórico anterior preservado e mesclado")
        except:
            print(f"[AVISO] Não foi possível carregar histórico anterior,
            criando novo")

    with open(history_path, 'w') as f:
        json.dump(history, f, indent=2)
    print(f"\nHistórico salvo em: {history_path}")
```

```
    print(f"\n{'='*60}")
    print("TREINAMENTO CONCLUÍDO!")
    print(f"{'='*60}")
    print(f"Melhor modelo (val loss): {save_dir / 'best_model.pt'}")
    print(f"  Época: {best_epoch}, Val Loss: {best_val_loss:.4f}")
    print(f"\nMelhor modelo SEM OVERFITTING: {save_dir
    / 'best_no_overfitting_model.pt'}")
    print(f"  Época: {best_no_overfitting_epoch},
    Val Loss: {best_no_overfitting_val_loss:.4f}")
    print(f"  Score combinado: {best_no_overfitting_score:.4f}")
    print(f"\nÚltimo modelo: {save_dir / 'last_model.pt'}")
    print(f"Checkpoints: {save_dir / 'checkpoint_epoch_*.pt'}")


if __name__ == "__main__":
    parser = argparse.ArgumentParser
    (description="Treinar Quantum Transformer Optimizer v5.0")
    parser.add_argument
    ("--epochs", type=int, default=50, help="Número de épocas")
    parser.add_argument
    ("--batch-size", type=int, default=4, help="Tamanho do batch")
    parser.add_argument(
    "--learning-rate", type=float, default=1e-4,
    help="Taxa de aprendizado")
    parser.add_argument
    ("--save-dir", type=str, default="models", help="Diretório para salvar modelos")
    parser.add_argument
    ("--resume-from", type=str, default=None,
    help="Caminho do checkpoint para continuar")
    parser.add_argument
    ("--num-circuits", type=int, default=50,
    help="Número de circuitos para treinar")
    parser.add_argument
    ("--device", type=str, default=None,
    help="Dispositivo (cuda/cpu)")
    parser.add_argument
    ("--patience", type=int, default=15,
    help="Early stopping: épocas sem melhoria (None = desabilitado)")
    parser.add_argument
    ("--min-delta", type=float, default=0.001,
    help="Melhoria mínima considerada significativa")

    args = parser.parse_args()

    train(
        num_epochs=args.epochs,
        batch_size=args.batch_size,
        learning_rate=args.learning_rate,
        save_dir=Path(args.save_dir),
        resume_from=Path(args.resume_from) if args.resume_from else None,
        num_circuits=args.num_circuits,
```

```
        device=args.device,
        patience=args.patience if args.patience > 0 else None,
        min_delta=args.min_delta
    )
```

APÊNDICE F – test_real_hardware_ibm.py

```
import argparse
import sys
from pathlib import Path
from typing import List, Dict, Optional
import json
from datetime import datetime
import time
import numpy as np

sys.path.append(str(Path(__file__).parent / "src"))

import torch
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_histogram

try:
    from qiskit_ibm_runtime import QiskitRuntimeService, Sampler
    IBM_AVAILABLE = True
except (ImportError, AttributeError) as e:
    IBM_AVAILABLE = False
    print("[ERRO] qiskit-ibm-runtime não disponível.")
    print(f"       Erro: {e}")
    print("")
    print("SOLUÇÕES:")
    print("1. Instalar qiskit-ibm-runtime:")
    print("   pip install qiskit-ibm-runtime")
    sys.exit(1)

from src.tokenizer import QuantumCircuitTokenizer
from src.model import QuantumTransformer
from src.metrics import OptimizationMetrics
from src.validator import CircuitValidator


def create_ghz_circuit(num_qubits: int) -> QuantumCircuit:
    qc = QuantumCircuit(num_qubits)
    qc.h(0)
    for i in range(num_qubits - 1):
        qc.cx(i, i+1)
    return qc


def create_qft_circuit(num_qubits: int) -> QuantumCircuit:
    from qiskit.circuit.library import QFT
```

```
    return QFT(num_qubits=num_qubits, do_swaps=True)


def create_qaoa_circuit(num_qubits: int) -> QuantumCircuit:
    qc = QuantumCircuit(num_qubits)
    for i in range(num_qubits):
        qc.h(i)
    for i in range(num_qubits - 1):
        qc.cx(i, i+1)
        qc.rz(np.pi/4, i+1)
        qc.cx(i, i+1)
    return qc


def get_expected_state(circuit_name: str) -> Optional[str]:
    if circuit_name.startswith("ghz"):
        num_qubits = int(circuit_name.split("_")[1])
        return "0" * num_qubits
    elif circuit_name.startswith("qft"):
        num_qubits = int(circuit_name.split("_")[1])
        return "0" * num_qubits
    elif circuit_name.startswith("qaoa"):
        num_qubits = int(circuit_name.split("_")[1])
        return "0" * num_qubits
    return None


def calculate_real_noise(counts: dict, expected_state: Optional[str] = None)
-> float:
    if not counts:
        return 1.0

    total_shots = sum(counts.values())
    if total_shots == 0:
        return 1.0

    if expected_state:
        expected_count = counts.get(expected_state, 0)
        return 1 - (expected_count / total_shots)
    else:
        max_count = max(counts.values())
        return 1 - (max_count / total_shots)


class RealHardwareTester:

    def __init__(
        self,
        model_path: Path,
        ibm_token: Optional[str] = None,
        backend_name: str = "ibm_kyoto",
```

```
        shots: int = 1024
    ):
        self.shots = shots
        self.device = "cuda" if torch.cuda.is_available() else "cpu"

        print("=" * 60)
        print("CARREGANDO MODELO v5.0")
        print("=" * 60)
        self.model, self.tokenizer = self._load_model(model_path)

        print("\n" + "=" * 60)
        print("CONECTANDO AO IBM QUANTUM")
        print("=" * 60)
        self.backend, self.service = self._connect_ibm(ibm_token, backend_name)

        self.metrics_calc = OptimizationMetrics()
        self.validator = CircuitValidator()
        self.simulator = AerSimulator()

        self.results = []

    def _load_model(self, model_path: Path):
        print(f"Carregando modelo de: {model_path}")

        if not model_path.exists():
            raise FileNotFoundError(f"Modelo não encontrado: {model_path}")

        checkpoint = torch.load(model_path, map_location=self.device)

        vocab_size = checkpoint.get("tokenizer_vocab_size", 113)
        print(f"Vocab size: {vocab_size}")

        tokenizer = QuantumCircuitTokenizer
        (max_qubits=128, max_circuit_length=500)

        model = QuantumTransformer(
            vocab_size=vocab_size,
            d_model=512,
            nhead=8,
            num_encoder_layers=6,
            num_decoder_layers=6,
            dim_feedforward=2048,
            max_seq_length=500,
            dropout=0.1,
            device=self.device
        )

        model.load_state_dict(checkpoint["model_state_dict"])
        model.to(self.device)
        model.eval()
```

```
        epoch = checkpoint.get("epoch", "N/A")
        val_loss = checkpoint.get("metrics", {}).get("val_loss", "N/A")
        print(f"[OK] Modelo v5.0 carregado
        (Época: {epoch}, Val Loss: {val_loss:.4f})")
        print(f"[OK] Device: {self.device}")

        return model, tokenizer

    def _connect_ibm(self, ibm_token: Optional[str], backend_name: str):
        try:
            try:
                service = QiskitRuntimeService(channel="ibm_quantum_platform")
                print(f"[OK] Conectado com credenciais salvas
                (ibm_quantum_platform)")
            except:
                try:
                    service = QiskitRuntimeService(channel="ibm_cloud")
                    print(f"[OK] Conectado com credenciais salvas (ibm_cloud)")
                except:
                    if ibm_token:
                        try:
                            QiskitRuntimeService.save_account(
                                channel=
                                "ibm_quantum_platform",
                                token=ibm_token,
                                overwrite=True
                            )
                            service = QiskitRuntimeService(channel=
                            "ibm_quantum_platform")
                            print(f"[OK] Token salvo e conectado")
                        except Exception as e:
                            QiskitRuntimeService.save_account(
                                channel=

                                "ibm_quantum_platform",
                                token=ibm_token
                            )
                            service = QiskitRuntimeService(channel=

                            "ibm_quantum_platform")
                            print(f"[OK] Token salvo e conectado")
                    else:
                        raise ValueError
                        ("Nenhuma conta salva encontrada.
                        Forneça token com --ibm-token")

            backends = service.backends()
            backend_names = [be.name for be in backends]

            print(f"\nBackends disponíveis ({len(backends)}):")
            for i, be in enumerate(backends[:10]):
```

```
                status = be.status()
                print(f"  {i+1}. {be.name} - {status.status_msg}
                ({be.num_qubits} qubits)")

            if backend_name not in backend_names:
                print(f"\n[AVISO] Backend '{backend_name}' não encontrado!")
                print(f"        Backends disponíveis:
                {', '.join(backend_names)}")

                if backends:
                    fallback_backend = backends[0]
                    print(f"\n[INFO] Usando backend alternativo:
                    {fallback_backend.name}")
                    backend_name = fallback_backend.name
                else:
                    raise ValueError(f"Nenhum backend disponível.
                    Backend solicitado: {backend_name}")

            backend = service.backend(backend_name)
            status = backend.status()

            print(f"\n[OK] Backend selecionado: {backend.name}")
            print(f"     Qubits: {backend.num_qubits}")
            print(f"     Status: {status.status_msg}")
            print(f"     Jobs na fila: {status.pending_jobs}")

            if status.status_msg != "active":
                print(f"\n[AVISO]
                Backend não está ativo. Status: {status.status_msg}")

            return backend, service

        except Exception as e:
            print(f"[ERRO] Erro ao conectar ao IBM Quantum: {e}")
            raise

    def optimize_with_transformer(self, circuit: QuantumCircuit)
    -> QuantumCircuit:
        print("  Otimizando com Transformer (v5.0)...")

        try:
            tokens = self.tokenizer.tokenize_circuit(circuit)

            src_tensor = torch.tensor([tokens], dtype=torch.long,

            device=self.device)

            self.model.eval()
            with torch.no_grad():
                generated_tokens = self.model.generate(
                    src=src_tensor,
```

```
                max_length=min(len(tokens) * 2, 1000),
                temperature=0.8,
                top_k=50,
                start_id=self.tokenizer.start_id,
                end_id=self.tokenizer.end_id
            )

        generated_list = generated_tokens[0].cpu().tolist()

        num_qubits = circuit.num_qubits
        optimized = self.tokenizer
        .detokenize_circuit(generated_list, num_qubits)

        if optimized.depth() == 0 and len(optimized.data) == 0:
            raise ValueError("Circuito gerado está vazio")

        if optimized.num_qubits > num_qubits:
            from qiskit import QuantumCircuit
            fixed_circuit = QuantumCircuit(num_qubits)
            for inst in optimized.data:
                qubits_used = [optimized.find_bit(q).index for q
                in inst.qubits if optimized.find_bit(q).index < num_qubits]
                if len(qubits_used) == len(inst.qubits):
                    fixed_circuit.append(inst.operation, qubits_used)
            optimized = fixed_circuit

        if optimized.num_qubits > self.backend.num_qubits:
            raise ValueError(f"Circuito muito grande:
            {optimized.num_qubits} > {self.backend.num_qubits}")

        optimized = transpile(
            optimized,
            backend=self.backend,
            optimization_level=0
        )

        if optimized.num_qubits > num_qubits:
            from qiskit import QuantumCircuit
            fixed_circuit = QuantumCircuit(num_qubits)
            for inst in optimized.data:
                qubits_used = [optimized.find_bit(q).index
                for q in inst.qubits if optimized.find_bit(q).index
                < num_qubits]
                if len(qubits_used) == len(inst.qubits):
                    fixed_circuit.append(inst.operation, qubits_used)
            optimized = fixed_circuit

        if optimized.num_qubits != num_qubits:
            raise
            ValueError(f"Número de qubits incorreto após otimização:
            {optimized.num_qubits} != {num_qubits}")
```

```
            print(f"    [OK] Circuito otimizado: qubits=
            {optimized.num_qubits},
            depth={optimized.depth()}")

            return optimized

        except Exception as e:
            print(f"    [ERRO] Erro ao usar modelo Transformer: {e}")
            print(f"    [FALLBACK] Usando Qiskit O3 como fallback...")
            return transpile(circuit, backend=self.backend,
            optimization_level=3)

    def optimize_with_qiskit(self, circuit: QuantumCircuit) -> QuantumCircuit:
        print("  Otimizando com Qiskit O3...")
        optimized = transpile(circuit, backend=self.backend,
        optimization_level=3)
        return optimized

    def execute_on_hardware(self,
    circuit: QuantumCircuit, original_num_qubits: Optional[int] = None) -> Dict:
        print(f"  Executando em {self.backend.name} ({self.shots} shots)...")

        if original_num_qubits is None:
            original_num_qubits = circuit.num_qubits

        circuit_with_measurements = circuit.copy()
        has_measurements = any(
            (hasattr(inst, 'operation') and hasattr(inst.operation, 'name')
            and inst.operation.name == 'measure')
            or
            (hasattr(inst, 'name')
            and inst.name == 'measure')
            for inst in circuit_with_measurements.data
        )

        if not has_measurements:
            if len(circuit_with_measurements.cregs) == 0:
                from qiskit import ClassicalRegister
                cr = ClassicalRegister(circuit_with_measurements.num_qubits,
                'c')
                circuit_with_measurements.add_register(cr)
            circuit_with_measurements.measure_all()

        transpiled = transpile(circuit_with_measurements, backend=self.backend,
        optimization_level=0)

        has_measurements_transpiled = any(
            (hasattr(inst, 'operation') and hasattr(inst.operation, 'name')
            and inst.operation.name == 'measure')
            or
```

```
            (hasattr(inst, 'name')
            and inst.name == 'measure')
            for inst in transpiled.data
        )
        has_cregs_transpiled = len(transpiled.cregs) > 0

        if not has_measurements_transpiled or not has_cregs_transpiled:
            if not has_cregs_transpiled:
                from qiskit import ClassicalRegister
                cr = ClassicalRegister(transpiled.num_qubits, 'c')
                transpiled.add_register(cr)
            if not has_measurements_transpiled:
                transpiled.measure_all()

        try:
            sampler = Sampler(mode=self.backend)
            print(f"  Submetendo job...")
            start_time = time.time()

            job = sampler.run([transpiled], shots=self.shots)
            job_id = job.job_id()
            print(f"  Job ID: {job_id}")
            print(f"  Aguardando resultados...")

            if transpiled.num_qubits <= 5:
                MAX_WAIT_TIME = 120
            elif transpiled.num_qubits <= 10:
                MAX_WAIT_TIME = 300
            else:
                MAX_WAIT_TIME = 600
            CHECK_INTERVAL = 10

            elapsed_wait = 0
            last_status = None

            while elapsed_wait < MAX_WAIT_TIME:
                try:
                    job_status = job.status()
                    status_name = job_status.name if hasattr(job_status, 'name')
                    else str(job_status)

                    if status_name != last_status:
                        print(f"  [INFO] Job status: {status_name} (aguardando
                        {elapsed_wait:.0f}s/{MAX_WAIT_TIME}s)")
                        last_status = status_name

                    if status_name in ['DONE', 'COMPLETED']:
                        print(f"  [OK] Job concluído! Obtendo resultado...")
                        result = job.result()
                        elapsed_time = time.time() - start_time
                        print(f"  [OK] Resultado obtido em {elapsed_time:.1f}s")
```

```
                    break
                elif status_name in ['ERROR', 'CANCELLED', 'FAILED']:
                    raise RuntimeError(f"Job {job_id} terminou com status:
                    {status_name}")

            except Exception as e:
                if elapsed_wait > 30:
                    try:
                        result = job.result()
                        elapsed_time = time.time() - start_time
                        print(f"  [OK] Resultado obtido em
                        {elapsed_time:.1f}s")
                        break
                    except:
                        pass

            elapsed_wait = time.time() - start_time

            if elapsed_wait >= MAX_WAIT_TIME:
                raise TimeoutError(f"Timeout após {MAX_WAIT_TIME}s")

            time.sleep(CHECK_INTERVAL)

        counts = {}
        try:
            if hasattr(result, '__len__') and len(result) > 0:
                pub_result = result[0]
                if hasattr(pub_result, 'data'):
                    pub_data = pub_result.data
                    if hasattr(pub_data, 'keys'):
                        keys = list(pub_data.keys())

                        if 'meas' in keys:
                            meas_value = pub_data['meas']
                            if hasattr(meas_value, 'get_counts'):
                                counts_dict = meas_value.get_counts()
                                if isinstance(counts_dict, dict):
                                    counts = dict(counts_dict)
                                elif hasattr(counts_dict, 'items'):
                                    counts = dict(counts_dict)

                        elif 'c' in keys and len(counts) == 0:
                            c_value = pub_data['c']
                            if hasattr(c_value, 'get_counts'):
                                counts_dict = c_value.get_counts()
                                if isinstance(counts_dict, dict):
                                    counts = dict(counts_dict)
                            elif isinstance(c_value, dict):
                                counts = dict(c_value)

                        elif 'quasi_dists' in keys and len(counts) == 0:
```

```
                                quasi_dists_list = pub_data['quasi_dists']
                                if quasi_dists_list and len(quasi_dists_list)
                                > 0:
                                    quasi_dist = quasi_dists_list[0]
                                    if hasattr(quasi_dist, 'items'):
                                        for bitstring, probability
                                        in quasi_dist.items():
                                            count = int(probability
                                            * self.shots)
                                            if count > 0:
                                                if isinstance(bitstring, int):
                                                    bitstring_str = format
                                                    (bitstring,
                                                    f'0{circuit.num_qubits}b')
                                                else:
                                                    bitstring_str =
                                                    str(bitstring)
                                                counts[bitstring_str] = count
        except Exception as e:
            print(f"  [ERRO] Erro ao extrair counts: {e}")
            counts = {"error": str(e)}

        if "error" not in counts and len(counts) > 0:
            truncated_counts = {}
            for bitstring, count in counts.items():
                if isinstance(bitstring, str):
                    if len(bitstring) > original_num_qubits:
                        truncated = bitstring[:original_num_qubits]
                    elif len(bitstring) < original_num_qubits:
                        truncated = bitstring + '0' * (original_num_qubits
                        - len(bitstring))
                    else:
                        truncated = bitstring

                    if truncated in truncated_counts:
                        truncated_counts[truncated] += count
                    else:
                        truncated_counts[truncated] = count
                else:
                    try:
                        if isinstance(bitstring, int):
                            truncated = format(bitstring,
                            f'0{original_num_qubits}b')
                        else:
                            truncated = str(bitstring)[:original_num_qubits]
                        if truncated in truncated_counts:
                            truncated_counts[truncated] += count
                        else:
                            truncated_counts[truncated] = count
                    except:
                        truncated_counts[str(bitstring)] = count
```

```
            counts = truncated_counts

        return {
            "backend": self.backend.name,
            "job_id": job_id,
            "counts": counts,
            "shots": self.shots,
            "execution_time": elapsed_time,
            "status": "success"
        }

    except Exception as e:
        print(f"  [ERRO] Erro ao executar: {e}")
        return {
            "error": str(e),
            "status": "error"
        }

def test_circuit(
    self,
    circuit: QuantumCircuit,
    circuit_name: str
) -> Dict:
    print("\n" + "=" * 60)
    print(f"TESTANDO: {circuit_name}")
    print("=" * 60)
    print(f"Qubits: {circuit.num_qubits}")
    print(f"Depth original: {circuit.depth()}")

    orig_metrics = self.metrics_calc.calculate_all_metrics(circuit)

    transformer_opt = self.optimize_with_transformer(circuit)
    transformer_metrics =
    self.metrics_calc.calculate_all_metrics(transformer_opt)
    transformer_improvements =
    self.metrics_calc.improvement_score(circuit, transformer_opt)
    transformer_validation =
    self.validator.validate_optimization(circuit, transformer_opt)

    qiskit_opt = self.optimize_with_qiskit(circuit)
    qiskit_metrics =
    self.metrics_calc.calculate_all_metrics(qiskit_opt)
    qiskit_improvements =
    self.metrics_calc.improvement_score(circuit, qiskit_opt)
    qiskit_validation =
    self.validator.validate_optimization(circuit, qiskit_opt)

    print("\nExecutando circuitos em hardware real...")
    original_num_qubits = circuit.num_qubits
    transformer_execution =
```

```
        self.execute_on_hardware(transformer_opt,
        original_num_qubits=original_num_qubits)
        time.sleep(2)
        qiskit_execution =
        self.execute_on_hardware(qiskit_opt,
        original_num_qubits=original_num_qubits)

        depth_diff =
        transformer_metrics['circuit_depth'] - qiskit_metrics['circuit_depth']
        gates_diff =
        transformer_metrics['total_gates'] - qiskit_metrics['total_gates']
        cx_diff =
        transformer_metrics['cx_gates'] - qiskit_metrics['cx_gates']
        fidelity_diff =
        transformer_metrics['expected_fidelity']
        - qiskit_metrics['expected_fidelity']

        transformer_multi_qubit =
        transformer_metrics.get('multi_qubit_gates', 0)
        qiskit_multi_qubit =
        qiskit_metrics.get('multi_qubit_gates', 0)
        multi_qubit_diff =
        transformer_multi_qubit - qiskit_multi_qubit

        transformer_swap =
        transformer_metrics.get('swap_gates', 0)
        qiskit_swap = qiskit_metrics.get('swap_gates', 0)
        swap_diff = transformer_swap - qiskit_swap

        transformer_time =
        transformer_execution.get('execution_time', 0)
        qiskit_time =
        qiskit_execution.get('execution_time', 0)
        time_diff = transformer_time - qiskit_time

        transformer_counts =
        transformer_execution.get('counts', {})
        qiskit_counts = qiskit_execution.get('counts', {})
        expected_state = get_expected_state(circuit_name)

        transformer_noise =
        calculate_real_noise(transformer_counts, expected_state)
        qiskit_noise =
        calculate_real_noise(qiskit_counts, expected_state)
        noise_diff = transformer_noise - qiskit_noise

        transformer_wins = 0
        qiskit_wins = 0

        if depth_diff < 0:
            transformer_wins += 1
```

```
        elif depth_diff > 0:
            qiskit_wins += 1

        if gates_diff < 0:
            transformer_wins += 1
        elif gates_diff > 0:
            qiskit_wins += 1

        if cx_diff < 0:
            transformer_wins += 1
        elif cx_diff > 0:
            qiskit_wins += 1

        if time_diff < 0:
            transformer_wins += 1
        elif time_diff > 0:
            qiskit_wins += 1

        if noise_diff < 0:
            transformer_wins += 1
        elif noise_diff > 0:
            qiskit_wins += 1

        if multi_qubit_diff < 0:
            transformer_wins += 1
        elif multi_qubit_diff > 0:
            qiskit_wins += 1

        if swap_diff < 0:
            transformer_wins += 1
        elif swap_diff > 0:
            qiskit_wins += 1

        winner = "Transformer" if transformer_wins > qiskit_wins else "Qiskit"
        if qiskit_wins > transformer_wins else "Tie"

        result = {
            "circuit_name": circuit_name,
            "num_qubits": circuit.num_qubits,
            "timestamp": datetime.now().isoformat(),
            "original": {
                "depth": orig_metrics["circuit_depth"],
                "total_gates": orig_metrics["total_gates"],
                "cx_gates": orig_metrics["cx_gates"],
                "expected_fidelity": float(orig_metrics["expected_fidelity"]),
                "multi_qubit_gates": orig_metrics.get("multi_qubit_gates", 0),
                "swap_gates": orig_metrics.get("swap_gates", 0)
            },
            "transformer": {
                "depth":
                transformer_metrics["circuit_depth"],
```

```
                "total_gates":
                transformer_metrics["total_gates"],
                "cx_gates":
                transformer_metrics["cx_gates"],
                "expected_fidelity":
                float(transformer_metrics["expected_fidelity"]),
                "depth_reduction":
                float(transformer_improvements["depth_reduction"]),
                "gates_reduction":
                float(transformer_improvements["gate_reduction"]),
                "cx_reduction":
                float(transformer_improvements["cx_reduction"]),
                "multi_qubit_gates":
                transformer_metrics.get("multi_qubit_gates", 0),
                "swap_gates":
                transformer_metrics.get("swap_gates", 0),
                "is_equivalent":
                bool(transformer_validation["is_equivalent"]),
                "execution": transformer_execution
            },
            "qiskit": {
                "depth":
                qiskit_metrics["circuit_depth"],
                "total_gates":
                qiskit_metrics["total_gates"],
                "cx_gates":
                qiskit_metrics["cx_gates"],
                "expected_fidelity":
                float(qiskit_metrics["expected_fidelity"]),
                "depth_reduction":
                float(qiskit_improvements["depth_reduction"]),
                "gates_reduction":
                float(qiskit_improvements["gate_reduction"]),
                "cx_reduction":
                float(qiskit_improvements["cx_reduction"]),
                "multi_qubit_gates":
                qiskit_metrics.get("multi_qubit_gates", 0),
                "swap_gates":
                qiskit_metrics.get("swap_gates", 0),
                "is_equivalent":
                bool(qiskit_validation["is_equivalent"]),
                "execution": qiskit_execution
            },
            "comparison": {
                "depth_diff": float(depth_diff),
                "gates_diff": float(gates_diff),
                "cx_diff": float(cx_diff),
                "fidelity_diff": float(fidelity_diff),
                "time_diff": float(time_diff),
                "noise_diff": float(noise_diff),
                "transformer_noise": float(transformer_noise),
```

```
                "qiskit_noise": float(qiskit_noise),
                "multi_qubit_diff": float(multi_qubit_diff),
                "swap_diff": float(swap_diff),
                "transformer_wins": transformer_wins,
                "qiskit_wins": qiskit_wins,
                "winner": winner,
                "victory_criteria":
                "v5.0: Depth, Gates, CX, Time, Noise, Multi-qubit, SWAP
                (7 critérios como v4.0)"
            },
            "gate_analysis": {
                "transformer": {
                    "all_gates":

                    transformer_metrics.get('all_gates', {}),
                    "single_qubit":
                    transformer_metrics.get('single_qubit_gates', 0),
                    "two_qubit":
                    transformer_metrics.get('two_qubit_gates', 0),
                    "multi_qubit":
                    transformer_metrics.get('multi_qubit_gates', 0),
                    "swap":
                    transformer_metrics.get('swap_gates', 0)
                },
                "qiskit": {
                    "all_gates": qiskit_metrics.get('all_gates', {}),
                    "single_qubit": qiskit_metrics.get('single_qubit_gates', 0),
                    "two_qubit": qiskit_metrics.get('two_qubit_gates', 0),
                    "multi_qubit": qiskit_metrics.get('multi_qubit_gates', 0),
                    "swap": qiskit_metrics.get('swap_gates', 0)
                }
            }
        }

        print("\n" + "-" * 60)
        print("RESUMO:")
        print(f"  Original:
        Depth={orig_metrics['circuit_depth']},
        Gates={orig_metrics['total_gates']},
        CX={orig_metrics['cx_gates']}")
        print(f"  Transformer:
        Depth={transformer_metrics['circuit_depth']}
        ({transformer_improvements['depth_reduction']:.1f}%),
        Gates={transformer_metrics['total_gates']}
        ({transformer_improvements['gate_reduction']:.1f}%),
        CX={transformer_metrics['cx_gates']}
        ({transformer_improvements['cx_reduction']:.1f}%)")
        print(f"  Qiskit:
        Depth={qiskit_metrics['circuit_depth']}
        ({qiskit_improvements['depth_reduction']:.1f}%),
        Gates={qiskit_metrics['total_gates']}
```

```
        ({qiskit_improvements['gate_reduction']:.1f}%),
        CX={qiskit_metrics['cx_gates']}
        ({qiskit_improvements['cx_reduction']:.1f}%)")
        print(f"  Tempo:
        Transformer={transformer_time:.2f}s,
        Qiskit={qiskit_time:.2f}s (diff: {time_diff:+.2f}s)")
        print(f"  Ruído Real:
        Transformer={transformer_noise:.4f}
        ({transformer_noise*100:.2f}%), Qiskit=
        {qiskit_noise:.4f} ({qiskit_noise*100:.2f}%) (diff:
        {noise_diff:+.4f})")
        print(f"  Multi-qubit:
        Transformer={transformer_multi_qubit}, Qiskit=
        {qiskit_multi_qubit} (diff: {multi_qubit_diff:+.0f})")
        print(f"  SWAP:
        Transformer={transformer_swap}, Qiskit={qiskit_swap}
        (diff: {swap_diff:+.0f})")
        print(f"  Vencedor (v5.0 - 7 critérios):

        {winner} (Transformer: {transformer_wins}/7, Qiskit: {qiskit_wins}/7)")
        print(f"     Critérios:
        Depth, Gates, CX, Time,
        Noise, Multi-qubit, SWAP")
        print(f"  Equivalência:
        Transformer=
        {transformer_validation['is_equivalent']},
        Qiskit={qiskit_validation['is_equivalent']}")
        print("-" * 60)

        return result

    def run_test_suite(
        self,
        circuit_types: List[str],
        sizes: List[int],
        output_path: Optional[Path] = None
    ) -> Dict:
        self.output_path = output_path

        print("\n" + "=" * 60)
        print("SUITE DE TESTES: Transformer vs Qiskit (Hardware Real) - v5.0")
        print("=" * 60)
        print(f"Backend: {self.backend.name}")
        print(f"Circuitos: {', '.join(circuit_types)}")
        print(f"Tamanhos: {sizes}")
        print(f"Shots: {self.shots}")
        print("v5.0: Critérios: Depth, Gates, CX, Time, Noise, Multi-qubit,
        SWAP (7 critérios como v4.0)")
        print("=" * 60)

        results = []
```

```
        for circuit_type in circuit_types:
            for size in sizes:
                try:
                    if circuit_type == "ghz":
                        circuit = create_ghz_circuit(size)
                    elif circuit_type == "qft":
                        circuit = create_qft_circuit(size)
                    elif circuit_type == "qaoa":
                        circuit = create_qaoa_circuit(size)
                    else:
                        print(f"[AVISO] Tipo de circuito desconhecido:
                        {circuit_type}")
                        continue

                    if circuit.num_qubits > self.backend.num_qubits:
                        print(f"[AVISO] Circuito {circuit_type}_{size}
                        tem {circuit.num_qubits} qubits, mas backend
                        tem apenas {self.backend.num_qubits}")
                        continue

                    circuit_name = f"{circuit_type}_{size}"
                    result = self.test_circuit(circuit, circuit_name)
                    results.append(result)

                    if self.output_path:
                        try:
                            incremental_data = {
                                "timestamp": datetime.now().isoformat(),
                                "backend": self.backend.name,
                                "shots": self.shots,
                                "circuit_types": circuit_types,
                                "sizes": sizes,
                                "results": results,
                                "progress": {
                                    "completed": len(results),
                                    "total_expected": len(circuit_types)
                                    * len(sizes)
                                }
                            }
                            try:
                                with open(self.output_path, 'w') as f:
                                    json.dump(incremental_data, f, indent=2,
                                    default=str)
                                print(f"[INFO] Resultados incrementais salvos
                                ({len(results)}/{len(circuit_types)
                                * len(sizes)})")
                            except:
                                pass
                        except:
                            pass
```

```
            except Exception as e:
                print(f"[ERRO] Erro ao testar {circuit_type} com
                {size} qubits: {e}")
                continue

    if results:
        transformer_wins = sum(1 for r in results if r['comparison']
        ['winner'] == 'Transformer')
        qiskit_wins = sum(1 for r in results if r['comparison']
        ['winner'] == 'Qiskit')
        ties = sum(1 for r in results if r['comparison']
        ['winner'] == 'Tie')

        avg_depth_improvement = np.mean([
            r['transformer']['depth_reduction']
            - r['qiskit']['depth_reduction']
            for r in results
        ])
        avg_cx_improvement = np.mean([
            r['transformer']['cx_reduction']
            - r['qiskit']['cx_reduction']
            for r in results
        ])

        print("\n" + "=" * 60)
        print("RESUMO GERAL (v5.0)")
        print("=" * 60)
        print(f"Total de testes: {len(results)}")
        print(f"Transformer venceu: {transformer_wins} ({transformer_wins
        /len(results)*100:.1f}%)")
        print(f"Qiskit venceu: {qiskit_wins} ({qiskit_wins/len(results)*100:.1f}%)")
        print(f"Empates: {ties} ({ties/len(results)*100:
        .1f}%)")
        print(f"\nMelhorias médias (Transformer vs Qiskit):")
        print(f"  Depth: {avg_depth_improvement:+.2f}%")
        print(f"  CX Gates: {avg_cx_improvement:+.2f}%")
        print(f"\nCritérios de vitória v5.0: Depth, Gates, CX, Time, Noise,
        Multi-qubit, SWAP (7 critérios como v4.0)")
        print("=" * 60)

    return {
        "timestamp": datetime.now().isoformat(),
        "backend": self.backend.name,
        "shots": self.shots,
        "circuit_types": circuit_types,
        "sizes": sizes,
        "summary": {
            "total_tests": len(results),
            "transformer_wins": transformer_wins if results else 0,
            "qiskit_wins": qiskit_wins
```

```
                if results else 0,
                "ties": ties if results else 0,
                "avg_depth_improvement": float(avg_depth_improvement)

                if results else 0.0,
                "avg_cx_improvement": float(avg_cx_improvement) if results else 0.0,
                "victory_criteria": "v5.0: Depth, Gates, CX, Time, Noise,
                Multi-qubit, SWAP (7 critérios como v4.0)"
            },
            "results": results
        }


def main():
    parser = argparse.ArgumentParser(
        description="Testar modelo Transformer v5.0 vs Qiskit em hardware
        real da IBM"
    )

    parser.add_argument("--model", type=str, default="models/best_model.pt",
                        help="Caminho do modelo treinado (v5.0)")
    parser.add_argument("--ibm-token", type=str, default=None,
                        help="Token do IBM Quantum (ou usar credenciais salvas)")
    parser.add_argument("--backend", type=str, default="ibm_fez",
                        help="Backend IBM a usar")
    parser.add_argument("--circuits", nargs="+", default=["ghz", "qft", "qaoa"],
                        help="Tipos de circuitos para testar")
    parser.add_argument("--sizes", nargs="+", type=int, default=[3, 4, 5, 6],
                        help="Tamanhos (número de qubits)")
    parser.add_argument("--shots", type=int, default=1024,
                        help="Número de shots para execução")
    parser.add_argument("--output", type=str,
    default="real_hardware_test_results_v5.json",
                        help="Arquivo de saída JSON")

    args = parser.parse_args()

    tester = RealHardwareTester(
        model_path=Path(args.model),
        ibm_token=args.ibm_token,
        backend_name=args.backend,
        shots=args.shots
    )

    output_path = Path(args.output)
    results = tester.run_test_suite(
        circuit_types=args.circuits,
        sizes=args.sizes,
        output_path=output_path
    )
```

```python
    with open(output_path, 'w') as f:
        json.dump(results, f, indent=2, default=str)

    print(f"\n[OK] Resultados salvos em: {output_path}")

    return results


if __name__ == "__main__":
    main()
```